A FAST VIEWER FOR AOE SIMULATION RESULT FILES

aoeViewer

Version 2.0.0

User and Reference Guide

The interface, the algorithms, the file format,
the C ABI and the scripting API



Compiled September 14, 2026

Contents

1 What aoeViewer is **1**

1.1 The shape of the program . . . 1

1.2 What the program does . . . 2

2 Getting started **3**

2.1 What you need . . . 3

2.2 Building and running . . . 3

2.2.1 Where the library is looked for . . . 3

2.3 Command-line options . . . 3

2.4 Logging, and what to do when it will not start . . . 4

2.5 The first plot . . . 4

3 The window **7**

3.1 Anatomy . . . 7

3.2 The main toolbar . . . 8

3.3 Mouse modes . . . 8

3.4 The status bar . . . 9

3.5 Files that change underneath you . . . 9

3.6 Theme . . . 9

4 The twelve panels **11**

4.1 DATA — choosing runs . . . 11

4.2 PARAMS — choosing the axes of a tile . . . 12

4.3 COMPUTE — channels the file does not contain . . . 13

4.4 ALIAS — a second name for a channel . . . 13

4.5 ISO — filtering runs by a value they hold constant . . . 14

4.5.1 Grouping values into intervals . . . 17

4.5.2 Two constraints, composed . . . 17

4.5.3 What a workspace keeps, and what it does when the data has moved on . . . 18

4.6 STYLE — colour, line, marker and legend . . . 19

4.6.1 What decides a curve's colour . . . 19

4.6.2 The legend . . . 21

4.7 SCALE — axis ranges and cross-tile constraints . . . 21

4.7.1 When two constraints meet on one axis . . . 23

4.8 3D — three channels against each other . . . 24

4.9 INSPECT — reading values off a curve . . . 25

4.10 AREA — shaded bands . . . 27

4.11 MARKERS — an instant, drawn two ways . . . 27

4.12 TEXT — captions on a tile . . . 27

5 Pages, grids and tiles 32
5.1 The page and tile bar . . . 32
5.2 How a page is modelled . . . 32
5.3 What a tile remembers . . . 34
5.4 Robustness when the files change . . . 34

6 Exporting, and saving your work 35
6.1 Image . . . 35
6.2 CSV . . . 35
6.3 Workspaces . . . 36
6.3.1 Save, and Save as . . . 36
6.3.2 Loading against data that has moved on . . . 36
6.4 Background work, and why the interface freezes when it does . . . 36

7 The AOE file format 38
7.1 Overall shape . . . 38
7.2 Two header dialects . . . 38
7.3 How runs are delimited . . . 39
7.4 Records, rows and wrapped lines . . . 39
7.5 Numeric encoding . . . 39
7.6 File extensions . . . 40

8 How the data is read 41
8.1 Memory mapping . . . 41
8.2 One index pass, then lazy parallel columns . . . 41
8.3 The inner loop . . . 42
8.4 What happens to malformed input . . . 42
8.5 The `.aoec` sidecar . . . 42
8.5.1 Layout . . . 43
8.5.2 Deciding whether it is still valid . . . 43
8.5.3 Where sidecars live . . . 43
8.5.4 What it buys . . . 44
8.6 The memory budget . . . 44

9 Computed parameters: the formula language 45
9.1 How an expression is executed . . . 45
9.2 Operators . . . 45
9.3 Element-wise functions . . . 45
9.3.1 The three logarithms . . . 46
9.4 Whole-series functions . . . 47

9.5 Constants and the lookup form . . . 47
9.6 Worked examples . . . 48
9.7 Names, dependencies and errors . . . 48

10 The algorithms behind the plots 50
10.1 Decimation: drawing what the screen can show . . . 50
10.1.1 Selecting the visible window . . . 50
10.1.2 The pixel budget . . . 50
10.1.3 Statistics over the same window . . . 51
10.2 The redraw cache . . . 51
10.3 ISO grouping . . . 51
10.3.1 Boundaries . . . 52
10.3.2 How values are written . . . 52
10.4 Legend order, and why it stopped shuffling . . . 52
10.5 Logarithmic axes . . . 53
10.6 The three-dimensional projection . . . 53

11 The C ABI 54
11.1 Handles, versioning and errors . . . 54
11.2 Thread safety . . . 54
11.3 Function reference . . . 55
11.4 Building against it . . . 59

12 The Python scripting API 60
12.1 The one rule . . . 60
12.2 Session . . . 60
12.3 Handles . . . 60
12.4 A worked script . . . 60

13 Performance, limits and troubleshooting 63
13.1 Where the time goes . . . 63
13.2 Getting the best out of it . . . 63
13.3 Troubleshooting . . . 63
13.3.1 The program will not start . . . 63
13.3.2 A file opens but shows nothing . . . 64
13.3.3 The curves look wrong on a phase plane . . . 64
13.3.4 A value I expected is missing or reads as *nan* . . . 64
13.3.5 A channel disappeared after I removed a computed parameter . . . 64
13.3.6 Colours changed when I opened another file . . . 64
13.3.7 I reopened a workspace and the plot is empty . . . 64

A Keyboard shortcuts and gestures 65

B Environment variables 66

C The workspace file 67

D Defaults at a glance 68

E Known limitations 69

List of Figures

2.1 The window at startup, with one small file loaded and no parameters chosen yet. The vertical strip on the left is the tab rail; the panel beside it belongs to the selected tab; the bar above the plot controls pages, the grid and the active tile. 5

2.2 Twenty-five altitude profiles, drawn from a 578 MB file. The **PARAMS** tab is open: the x axis is `TIME`, and `ALTITUDE` is ticked in the y list. The status bar reports four files, twenty-five runs plotted and thirty-nine parameters. The window is shown cropped at the right. 6

3.1 A 2×2 page. Top left is a three-dimensional trajectory; top right a computed channel, `CLIMB_RATE`; bottom left a channel reached through an alias, *Angle of attack*; bottom right the altitude trace, which is the active tile and therefore carries the brighter border. 7

3.2 The file actions at the left of the toolbar. **Save workspace** overwrites the file currently open; **Save workspace as** — the same icon with a plus — always asks for a path and then continues in the new file. 8

3.3 The status bar with an ISO filter active. Eight of the twenty-five ticked runs are admitted by **DATA** but excluded by **ISO**, and the bar says so rather than silently showing seventeen. 9

3.4 The same page in the light theme. The palette is defined as a pair of complete token sets rather than as a set of overrides, so every colour — panel, surface, border, accent, plot background, grid, selection — has an explicit value in both. 10

4.1 Left: the file and run tree, with one file collapsed and unticked and another fully ticked. Right: the context menu on a file row, which offers the same operations by name. 11

4.2 Runs 11 and 12 are ticked, but the ISO filter excludes their mass interval, so they are dimmed. Nothing about the **DATA** selection has changed — widening the filter brings them back without re-ticking anything. 12

4.3 The **PARAMS** panel with the filter `AL*` typed, which has reduced thirty-nine channels to the three whose names match. 13

4.4 Left: a name and an expression, with the validator's verdict beneath. Right: the list of defined parameters, each shown as `name = source`. 13

4.5 `CLIMB_RATE`, defined as `deriv(ALTITUDE, TIME)`, plotted for all twenty-five runs. It was never in the file: the core evaluates it per run, and the tile cannot tell the difference between it and a channel that was. 14

4.6 The **Formula reference** dialog. Operators, element-wise functions, the three logarithms, series functions, reductions, constants, the lookup form and a dozen worked examples. The logarithm section is reproduced in §9.3.1. 15

4.7 An alias being created. `ALPHA` keeps its name and gains a second one; both resolve to the same samples. 15

4.8 A tile plotted through an alias. The axis label, the legend row and the tile title all read *Angle of attack*; the samples are `ALPHA`. 16

4.9 Left: `INITIAL_MASS` added as a filter, ungrouped, showing all seventy-five distinct values with the badge *all 75*. Right: the same parameter after grouping into three intervals, with the middle one unticked — the badge now reads *2/3*. 16

4.10 The grouping dialog. Seventy-five distinct masses between 57 236.3 and 79 012.1 have been split into three intervals; the *Values* column counts what falls in each, and the note underneath confirms that every value falls in exactly one group. 17

4.11 The middle mass interval has been unticked — the node now reads *2/3*. Eight of the twenty-five ticked runs are held back and their curves are gone; the **DATA** selection is untouched, and re-ticking the interval restores them. The status bar states both numbers. The window is shown cropped at the right. 18

4.12 Left: colouring by simulation, so the *Curves* table has one row per plotted run. Right: colouring by ISO, so it has one row per mass interval — three rows governing twenty-five curves. 20

4.13 Twenty-five curves, three legend rows. With **File** and **Simulation** off, identical rows are merged — which is precisely what turns a mass sweep into one row per mass interval. 21

4.14 The **SCALE** panel with the x axis locked. The four spin boxes are enabled only while their axis is locked; the *Constrained variables* list shows that `TIME` now carries a range attached to the variable itself. 22

4.15 The **3D** panel. Every axis and view control is disabled until **Draw the active tile in 3D** is ticked. 24

4.16 Twenty-five trajectories: ground track east against north, with altitude as z. The grid is drawn on the three faces of the data box that lie *behind* the trajectory from the current viewpoint, and swaps faces as the object turns. The triad in the corner keeps a constant size however far you zoom. 25

4.17 Left: every channel of one run at one instant, read after clicking a curve in **Inspect** mode. Right: per-curve statistics over the visible x window. 26

4.18 A slope measured by dragging across the climb. The gradient, both deltas, the two endpoints and the reciprocal are all reported; the reciprocal is there because on a time axis it is the quantity you usually want next. 26

4.19 The **AREA** panel. Bands are global: they appear on every page, on every tile whose axes can express them. 27

4.20 One band, three tiles. Every tile whose x axis is `TIME` shows it as a shaded region; the three-dimensional tile shows nothing, and the band is drawn *in front of* the curves, because a band underneath them lost its edges wherever a curve crossed one. 28

4.21 The **MARKERS** panel. **Dot size** governs the second of the two renderings described below. 28

4.22 The marker as a dashed vertical line, on the three tiles whose x axis is the marker's own channel. 29

4.23 Angle of attack against altitude, with the same marker and the same band as the previous figures. The marker is now a red dot on each curve at the instant `TIME = 707.35`; the band is the thick blue stretch of each curve during `TIME` 580–854. On a trajectory this answers a question a vertical stripe cannot: *where was the aircraft during those ten seconds?* 29

4.24 The **TEXT** editor. The nine **Place** buttons are corner and edge presets; **X** and **Y** are fractions of the tile, so a caption stays put through pan and zoom. 30

4.25 A caption placed at top centre of the altitude tile, over the shaded band it names. . . 31

5.1 The grid picker, mid-sweep. Cells whose layout would exceed twenty-five tiles are painted faint and cannot be chosen. 32

6.1 A page exported with **Image**. This figure is the file the button wrote, not a screenshot of the window — which is what makes it a useful way to get a figure out of the program and into a report. 35

CHAPTER 1

What aoeViewer is

aoeViewer is a desktop viewer for AOE simulation result files. It exists because the files are large — a single result file in routine use is half a gigabyte and holds tens of runs, each with dozens of channels and hundreds of thousands of samples — and because the questions asked of such files are interactive ones: which run overshot, what the load factor was at the moment of the pitch input, how the twenty-five cases in a mass sweep differ from one another. A tool that answers those questions has to redraw faster than the eye notices, and it has to do so while the data is far too large to hand to a plotting library sample by sample.

The program is built in two halves that meet at a deliberately narrow seam. The parsing, the computed-parameter engine and the plot decimation are C++17, compiled into a single shared library. The user interface is Python with Qt, by way of PySide6, and draws with pyqtgraph. Between them sits a plain C ABI: sixty exported functions, no C++ types, no Python headers. The consequence is that the core has no build-time dependency on Python at all — any CPython 3.8 or later can load it through `ctypes` — and that the same library serves the graphical application and a scripting session equally well.

1.1 The shape of the program

Layer	Responsibility
`core/` (C++17)	Memory-maps the result file, builds a sample index, parses columns on demand across a thread pool, evaluates computed parameters, decimates series for display, buckets ISO values into intervals, and reads and writes the `.aoec` sidecar. Owns all sample memory and the cache budget.
`core/include/aoe/aoe_c_api.h`	The seam: an opaque-handle C ABI with a thread-local error string. Everything above this line is written without a C++ compiler.
`python/aoeviewer/_core.py`	Discovers and loads the shared library through `ctypes`, and wraps returned pointers in zero-copy NumPy views whose lifetime is pinned to the underlying block.
`python/aoeviewer/session.py`	A Pythonic façade — `Session`, `FileHandle`, `SimulationHandle`, `IsoGroup` — which is also the public scripting API.
`python/aoeviewer/model.py`	`AppState`: everything the core does not own. Pages, tiles, selections, ISO filter definitions, styles, annotations, scale constraints, and the workspace file format. Contains no Qt.
`python/aoeviewer/ui/`	The Qt layer: the main window, the twelve side panels, the plot grid and the plot tiles.

Table 1.1: The layers, from the disk upwards.

A rule visible throughout the source is that anything testable without a display lives in a Qt-free module. `model.py`, `scales.py`, `isogroups.py` and `projection.py` import no Qt, which is why the decimation budget, the log-axis conversion, the interval arithmetic and the orthographic projection can all be tested headlessly, and are.

1.2 What the program does

Four ideas account for most of the interface.

A page of tiles. The plotting area is a grid of up to twenty-five independent tiles, arranged from 1×1 to 5×5, and there may be any number of pages. Each tile is a complete plot definition: its own x channel, its own set of y channels, its own legend, its own axis locks, its own colouring rule, and optionally its own three-dimensional projection.

Two independent constraints on what is drawn. The **DATA** tab chooses runs by name. The **ISO** tab chooses runs by the value some parameter holds constant across a run — a mass, a Mach number, a configuration. The two compose with a logical AND: a curve is drawn if and only if its run is ticked in **DATA** *and* admitted by every active **ISO** filter. Neither tab ever writes into the other's state.

Channels that need not exist in the file. A computed parameter is an expression over the channels that are there — `deriv(THETA, TIME)`, `NZ / max(NZ)`, `ALTITUDE[TIME=last] - ALTITUDE[TIME=first]` — evaluated in the core, per run, and thereafter indistinguishable from a channel the file carried. An alias is a second name for an existing channel, so a cryptic column can be given a readable one without touching the data.

Drawing what the screen can show, exporting what the data says. A tile with twenty-five runs of five hundred thousand samples does not hand two and a half million points to Qt. It asks the core for a min/max decimation of exactly the visible window at roughly two points per horizontal pixel, so that narrow spikes survive but the item count stays in the low thousands. The CSV export does the opposite and writes every sample, because a plot is a picture and an export is data.

> Every label, default, range and limit quoted here was read from the source of version 2.0.0, and every screenshot was taken from a running build with four result files open — one small sample and three 578 MB flight files of twenty-five runs each. Where the code and the documentation disagreed, the code won and the discrepancy is recorded in a box like this one. Where a genuine limitation exists, it is recorded in a red box.

CHAPTER 2

Getting started

2.1 What you need

Building the core requires nothing but a C++ compiler — GCC 8 or later, Clang 7 or later, MSVC 2019, or MinGW-w64 — and GNU `make` (or `mingw32-make.exe` on Windows). CMake is supported as an alternative but is not required. Running the graphical application additionally needs three Python packages: `numpy`, for array views over the core's buffers; `PySide6-Essentials` together with `shiboken6`, for the Qt widgets; and `pyqtgraph`, for the plotting canvas. Building the standalone executable adds `PyInstaller`.

2.2 Building and running

```
# 1. Build the core
./build.sh                 # Linux and macOS
build.bat                  # Windows (MinGW)

# 2. Check that it works
./build.sh test            # or: build.bat test

# 3. Run
PYTHONPATH=python python -m aoeviewer            # Linux and macOS
set PYTHONPATH=python && python -m aoeviewer     # Windows
```

The build produces `build/libaoecore.so` on Linux, `build/libaoecore.dylib` on macOS and `build/aoecore.dll` on Windows. Two directories are generated rather than kept in the repository: `build/`, holding the compiled core and the PyInstaller work files, and `dist/`, holding the frozen executable.

To try the program without real data, generate a sample:

```
python tools/gen_sample.py sample.aoe --sims 6 --rows 40000
PYTHONPATH=python python -m aoeviewer sample.aoe
```

2.2.1 Where the library is looked for

`_core.library_path()` searches in a fixed order, most specific first: the environment variable `AOEVIEWER_CORE` (which may name either a file or a directory); `sys._MEIPASS` and `sys._MEIPASS/aoeviewer`, which is where a PyInstaller bundle unpacks; the directory of the executable when frozen; the package directory; `<project>/build`; and finally the working directory. If none of them yields a loadable library, the raised `CoreError` lists every directory that was searched and suggests building the core first. On Windows a load failure that looks like a missing dependency adds a hint to run `objdump -p "<path>" | findstr "DLL Name"`.

2.3 Command-line options

Four options are recognised, and two of them are diagnostic tools worth knowing before you need them.

Option	Effect
`files...`	Result files to open at startup, given as positional arguments.
`-version`	Prints the application version, the core version, the path of the core library actually loaded, the worker count and the log path.
`-selftest`	Imports `numpy`, `PySide6`, `PySide6.QtWidgets` and `pyqtgraph` one at a time, loads the core through `ctypes`, then constructs a `QApplication`, falling back to the offscreen platform when there is no display. Prints `OK` or `FAIL` per component, then either `All checks passed.` or a count of failures. This is the first thing to run when the program will not start.
`-memory-limit MB`	Cap on cached sample data, default `2048.0`, floored at 64 MB. Columns beyond the cap are dropped least-recently-used first and re-read on demand.

Table 2.1: Options accepted by `python -m aoeviewer` and by the frozen executable.

2.4 Logging, and what to do when it will not start

The logger is named `aoeviewer` and writes `aoeviewer.log` through a rotating handler of 2 MB with three backups. The directory is chosen in order: the environment variable `AOEVIEWER_LOG_DIR`; the directory of the executable when frozen, or the project root when running from source; `~/.aoeviewer`; and the working directory. On startup the log records the application and core versions, the core library path, the Python and platform versions, the worker count and the PySide6, pyqtgraph and NumPy versions — which between them identify almost every environment problem.

The application installs `sys.excepthook`, `threading.excepthook` and a Qt message handler, so an unhandled exception anywhere, including inside a Qt callback, reaches the log rather than the void. If construction of the main window fails, the traceback is logged and printed, and a dialog titled **aoeViewer could not start** appears with the log path as its informative text and the traceback as its detail.

2.5 The first plot

When the program opens with no arguments the window is empty: a grid of one tile labelled *empty tile*, and a **DATA** tab with nothing in it.

Getting to a plot takes two steps. Press **Add files** in the **DATA** tab and choose one or more result files; each appears as a top-level row with its runs beneath it, and every run is ticked by default. Then open the **PARAMS** tab, where the x axis has already defaulted to the first channel whose name begins with `TIME`, and tick one or more y channels. The tile draws as soon as a tile has both an x channel and at least one y channel.

Opening a file of any size is done on a worker thread, and for files of 8 MB or more the same task also writes an `.aoec` sidecar beside the result file. That happens once; subsequent opens of the same file skip the index scan entirely and are several times faster. Chapter 8 describes the mechanism.



Figure 2.1: The window at startup, with one small file loaded and no parameters chosen yet. The vertical strip on the left is the tab rail; the panel beside it belongs to the selected tab; the bar above the plot controls pages, the grid and the active tile.

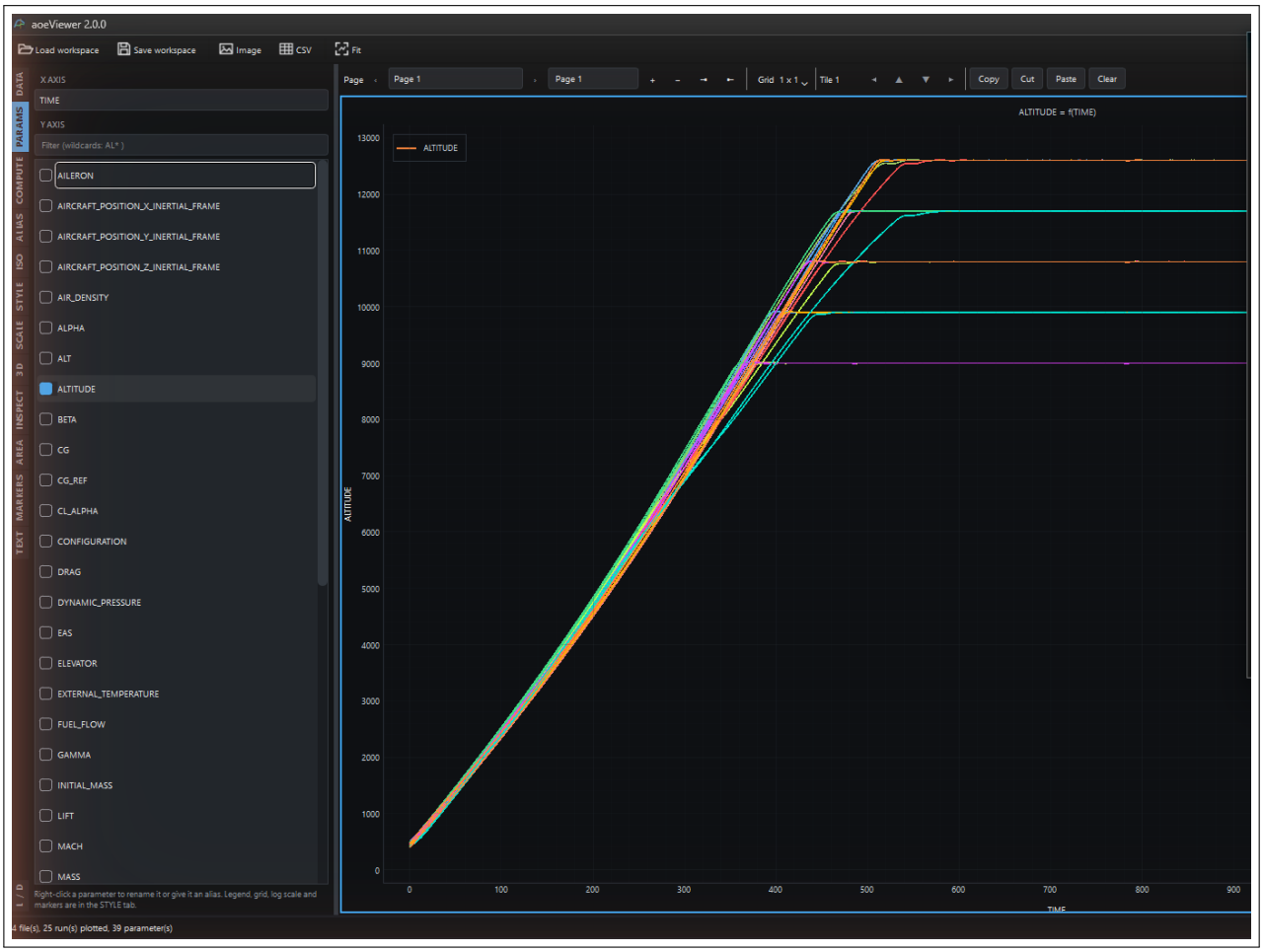


Figure 2.2: Twenty-five altitude profiles, drawn from a 578 MB file. The **PARAMS** tab is open: the x axis is `TIME`, and `ALTITUDE` is ticked in the y list. The status bar reports four files, twenty-five runs plotted and thirty-nine parameters. The window is shown cropped at the right.

CHAPTER 3

The window

3.1 Anatomy

The window has six regions. Across the top is the main toolbar, holding workspace load, save and save-as, the two exports, the auto-fit button and — pushed to the right edge — the four mouse modes. The title bar above it names the workspace currently open, with a leading asterisk while there are unsaved changes. Down the left edge runs the tab rail, a 21-pixel strip of vertically painted labels which selects which of the twelve panels is shown beside it. The panel itself occupies a splitter pane that can be dragged between 230 and 600 pixels wide. To its right, the plot area is divided into the page and tile bar along its top and the grid of tiles below. The status bar along the bottom carries, from left to right, the description of any running background task or the standing summary of what is loaded, an indeterminate progress bar while a task runs, the crosshair readout, and the size of the sample cache.

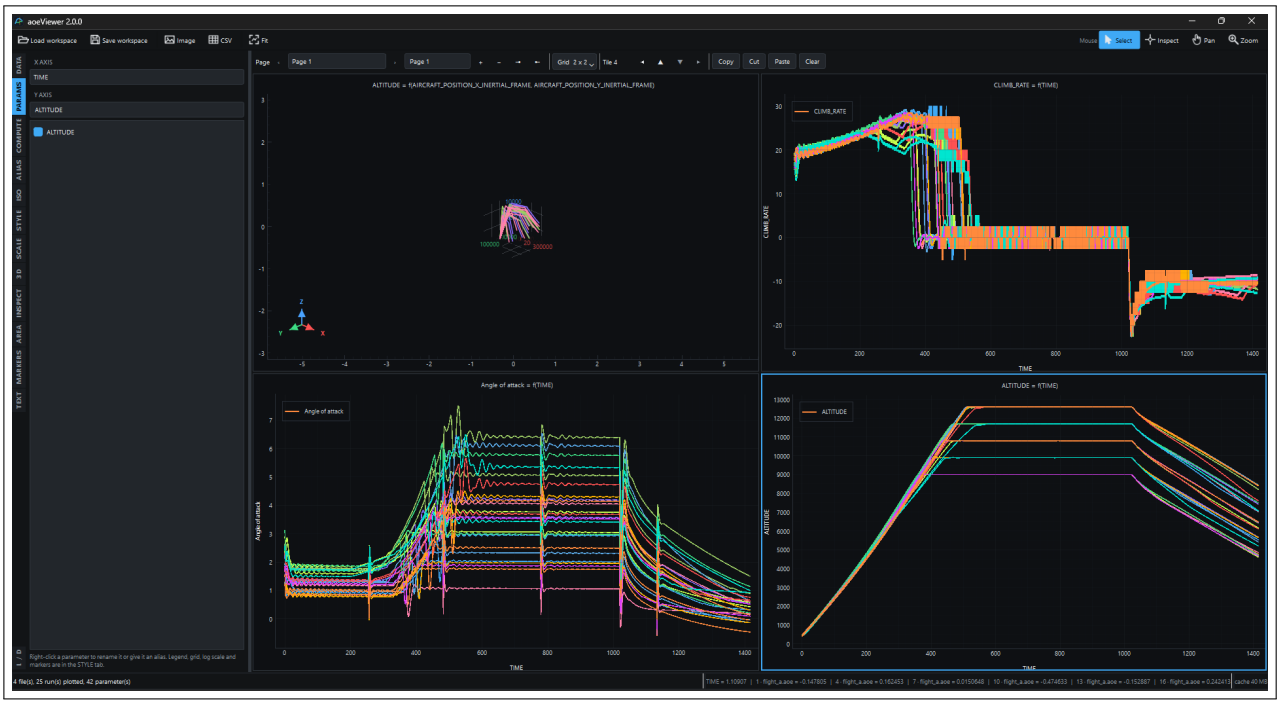


Figure 3.1: A 2×2 page. Top left is a three-dimensional trajectory; top right a computed channel, `CLIMB_RATE`; bottom left a channel reached through an alias, *Angle of attack*; bottom right the altitude trace, which is the active tile and therefore carries the brighter border.

The active tile is the one every per-tile control acts on, and it is marked by a two-pixel accent border rather than a one-pixel neutral one. Clicking any tile makes it active. That border is set through a guard that returns early when the state has not changed, which is not a micro-optimisation: `QWidget.setStyleSheet` performs no comparison of its own and repolishes the widget and all its children, and since a tile is a `QGraphicsView` that means a full scene repaint. Without the guard, one click on a 5×5 page would repaint twenty-five plots.

3.2 The main toolbar

Nine buttons in three groups: the workspace, the two exports, and auto-fit — then, pushed to the right edge, the four mouse modes. Every one of them is a labelled icon, and every icon is painted at request time rather than loaded from a file.

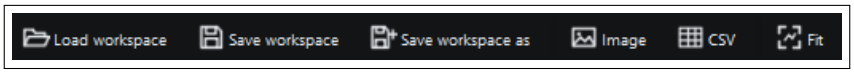


Figure 3.2: The file actions at the left of the toolbar. **Save workspace** overwrites the file currently open; **Save workspace as** — the same icon with a plus — always asks for a path and then continues in the new file.

Button	Shortcut	What it does
Load workspace	Ctrl+L	Opens a saved `.aoews` workspace, replacing everything currently loaded.
Save workspace	Ctrl+S	Overwrites the current workspace, asking for a path only if there is not one yet. The tooltip names the file it would write.
Save workspace as...	Ctrl+Shift+S	Always asks for a path, then continues in the new file and leaves the old one as it was.
Image	—	Saves the active page as a PNG, exactly as displayed.
CSV	—	Writes the active tile's curves to a CSV file at full resolution.
Fit	Ctrl+0	Auto-scales every tile, honouring axis locks. Double-clicking one plot fits just that one.
Select	Ctrl+1	Mouse mode: click to activate a tile; drag annotations.
Inspect	Ctrl+2	Mouse mode: click a curve to read every channel; drag to measure a slope.
Pan	Ctrl+3	Mouse mode: drag to move the view.
Zoom	Ctrl+4	Mouse mode: wheel zooms, drag a rectangle to zoom into it.

Table 3.1: The main toolbar. There is deliberately no **Open** action: **Add files** in the **DATA** tab and **Load workspace** here already cover it.

The toolbar is permanent by construction. Its context menu is suppressed, its `toggleViewAction` is disabled and hidden, and `MainWindow.createPopupMenu` returns nothing, because hiding the only toolbar in a window that has no menu bar would leave no way to bring it back. Its icons are not image files: they are painted on demand by `ui/icons.py` onto a 100×100 canvas and cached by name, colour and size, so they follow the theme and survive a PyInstaller bundle that ships no assets.

3.3 Mouse modes

The four modes are exclusive, and the inactive ones are genuinely off rather than merely ignored — in **Select** and **Inspect** the view box has mouse interaction disabled on both axes, and the wheel event is swallowed. Switching mode writes a one-line hint into the status bar.

Switching into **Inspect** brings the **INSPECT** panel to the front, and clicking a curve brings it forward again every time, because that is where the answer appears. Leaving **Inspect** clears the inspection dot. A locked axis stays locked whatever the mode says: the limits are re-applied and per-axis mouse interaction disabled on every redraw, so no drag or wheel gesture can escape a range you have frozen.

Mode	Drag does	Wheel	Annotations
Select	Moves markers and the edges of shaded areas; moves the legend and text captions.	inert	movable
Inspect	Rubber-bands a slope line; a drag shorter than six pixels counts as a click and inspects instead.	inert	frozen
Pan	Pans the view. On a 3D tile: turns it; middle-drag or Shift+drag slides it; right-drag or Ctrl+drag zooms.	inert	frozen
Zoom	Rubber-bands a rectangle to zoom into.	zooms	frozen

Table 3.2: What each mouse mode changes. Annotations are movable only in **Select** mode and only on a two-dimensional tile.

3.4 The status bar

The standing summary reads `N file(s), P run(s) plotted, M parameter(s).` When an ISO filter is holding runs back it becomes more explicit — `4 file(s), 17 of 25 run(s) plotted, 8 held back by the ISO filter(s), 42 parameter(s)` — which is the shortest honest description of a state that would otherwise look like lost data.

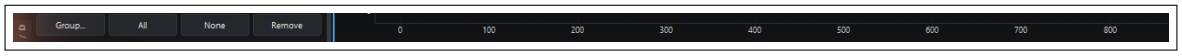


Figure 3.3: The status bar with an ISO filter active. Eight of the twenty-five ticked runs are admitted by **DATA** but excluded by **ISO**, and the bar says so rather than silently showing seventeen.

The progress bar is deliberately indeterminate: it has range 0 to 0, a fixed width of 120 pixels and no text. The core reports no progress from inside a parse, and a fabricated percentage would be worse than an honest *working*. On the right, `cache NN MB` is the sample data the core is holding; its tooltip explains that columns are dropped automatically when the figure exceeds the limit and re-read when needed again.

3.5 Files that change underneath you

A timer fires every three seconds and asks the core which open files have changed on disk, comparing path, size and modification time. It skips the check entirely while a background task is running, because the core session is not internally synchronised. When something has changed, a dialog titled **Files changed on disk** lists the names and offers to reload them. Declining is remembered against that exact stamp, so the question does not come back until the file changes again.

On closing, if anything has been modified, a three-way prompt titled **Unsaved changes** offers to save the workspace first; **Cancel** aborts the close. The application then waits up to thirty seconds for background work to finish before destroying the session, because tearing the core down while a worker is still inside it would write into freed memory.

3.6 Theme

At the foot of the tab rail sits a button labelled **L / D** whose tooltip reads *Switch between light and dark*. It flips the whole application, including the plot canvases, the legend grounds, the painted toolbar icons and the text captions.

The stylesheet sets every colour it depends on instead of inheriting from the desktop, because Qt's default palette differs enough between Linux and Windows that an application which is correct on one can be unreadable on the other. Two details are worth knowing. Check and radio indicators are drawn explicitly, including the ones inside tree, list and table views — those are drawn by the views

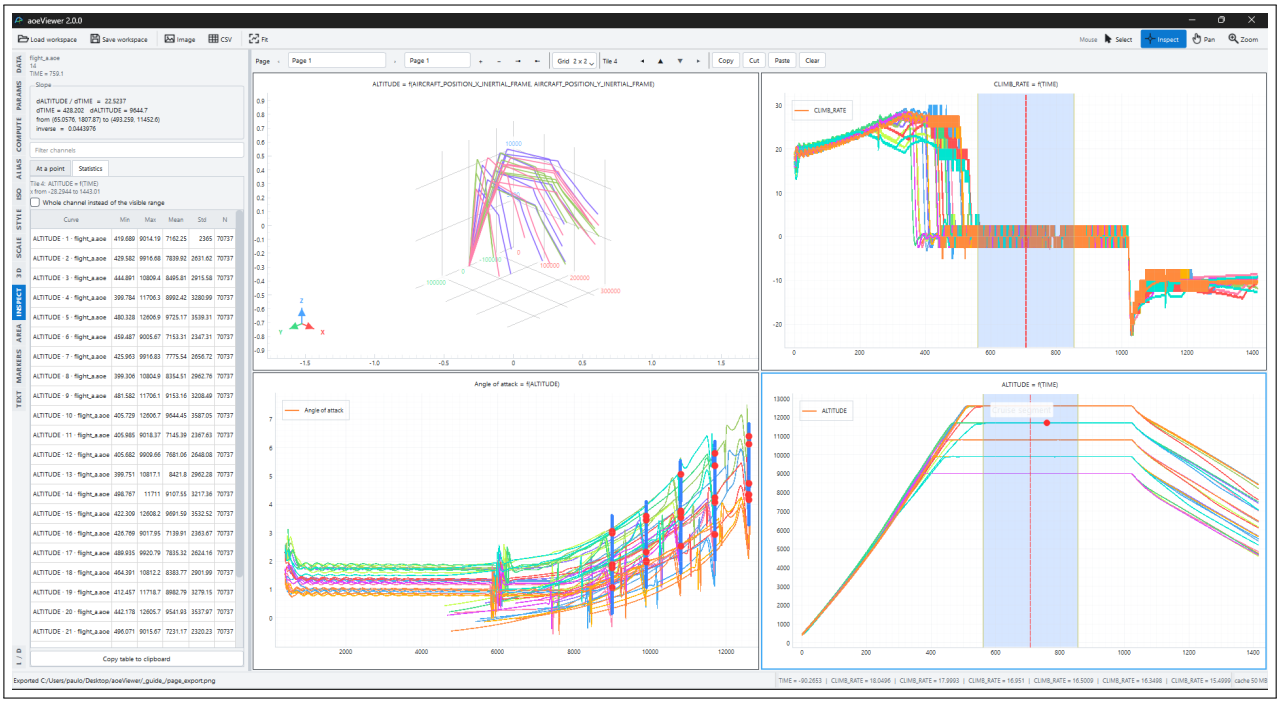


Figure 3.4: The same page in the light theme. The palette is defined as a pair of complete token sets rather than as a set of overrides, so every colour — panel, surface, border, accent, plot background, grid, selection — has an explicit value in both.

rather than by `QCheckBox`, and they are the ticks beside a run in **DATA**, a parameter in **PARAMS** and a value in **ISO**. And the type scale exists only as a set: widget text is sized in pixels (10 px body, 9 px for hints, section titles and the status bar) while plot text is sized in points (7.5 pt tile titles, 7 pt axes and legends, 6.5 pt readouts and captions), because a stylesheet that sets a pixel size makes `QFont.pointSizeF()` report -1 and any code that reads a font back would have to guess.

CHAPTER 4

The twelve panels

The tab rail selects one of twelve panels. Selecting a tab refreshes that panel immediately and marks the others stale, so they are refreshed lazily when next shown — which is why no panel is allowed to *write* application state while refreshing. Four of the panels act on the active tile only (**PARAMS**, **STYLE**, **SCALE**, **3D**, **TEXT**); the rest are global.

4.1 DATA — choosing runs

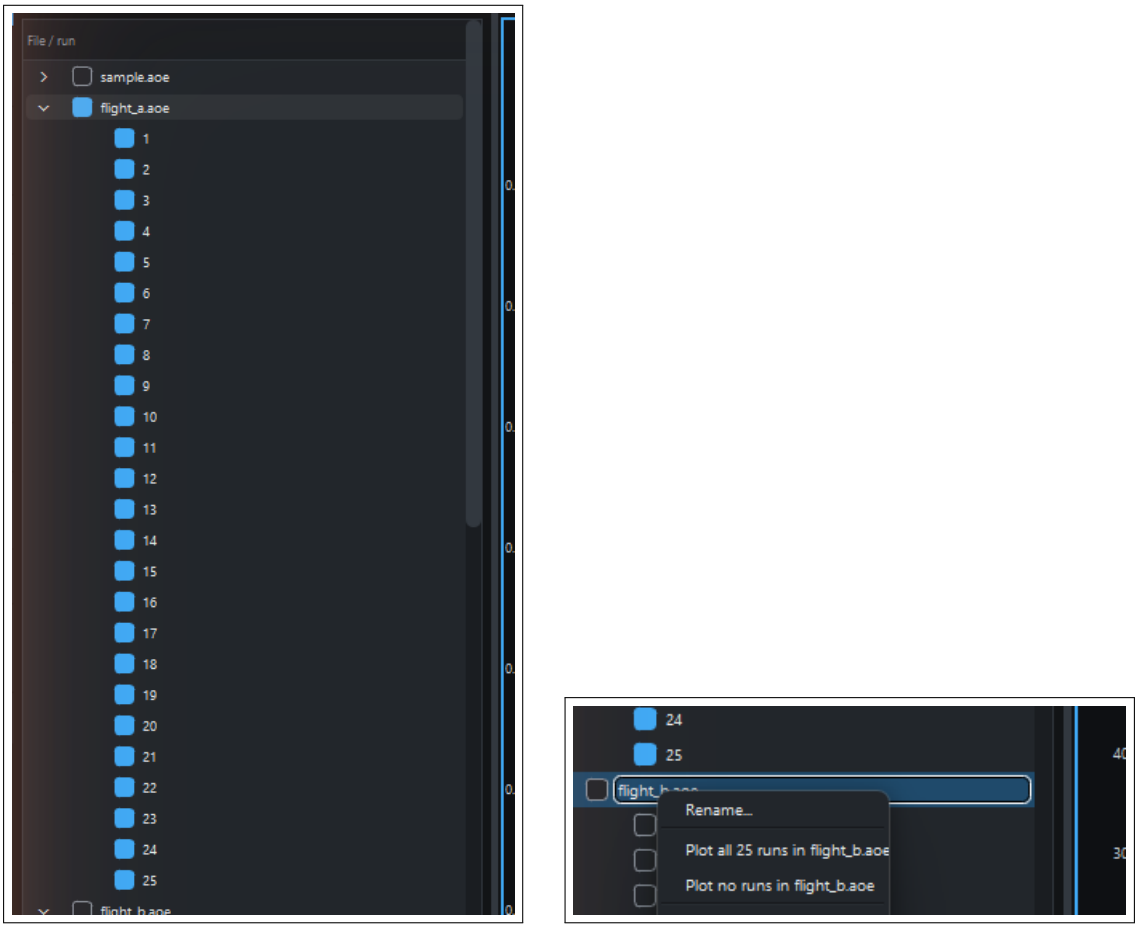


Figure 4.1: Left: the file and run tree, with one file collapsed and unticked and another fully ticked. Right: the context menu on a file row, which offers the same operations by name.

The panel is a tree with one top-level row per open file and one child row per run. **Add files** opens the file dialog, which filters on `AOE result files (*.aoe *.dat *.txt)`. **Remove** unloads the selected files, and acts only on file rows — selecting a single run and pressing **Remove** must not unload the whole file it belongs to. **Clear all** unloads everything.

Ticking a run plots it. Ticking a *file* row plots every run in that file, and clearing it plots none of them; the box is tri-state, so a file with only some of its runs ticked shows the indeterminate mark. The same two operations are on the context menu by name — **Plot all 25 runs in flight_b.aoe** and **Plot no runs in flight_b.aoe** — along with **Rename...** and, on a file row, **Copy full path**. At the foot of

the panel, **Plot all** and **Plot none** act across every loaded file, driven from the session rather than from the tree.

Renaming, by double-click or from the menu, changes the display name only. The identity a style attaches to is a separate key that never changes, so recolouring does not follow a rename; the tooltip of a renamed run records what the file calls it.

> The tree is rebuilt only when its *structure* changes — a signature over each file's index, name, path and the index, name and file-level identifier of each of its runs. Ticking a box leaves every widget in place and writes only the check states, so scroll position and expansion survive. Before this, selecting a run rebuilt the whole tree, which read as a flicker and lost your place in a twenty-five-run file.

A run that is ticked here but excluded by an ISO filter is painted grey, and its tooltip gains a sentence explaining that it is held back rather than unselected. That greying is the visible half of the two-constraint model described in §4.5.

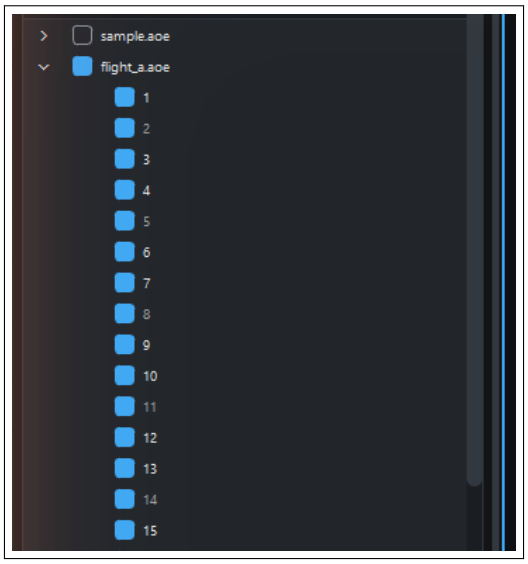


Figure 4.2: Runs 11 and 12 are ticked, but the ISO filter excludes their mass interval, so they are dimmed. Nothing about the **DATA** selection has changed — widening the filter brings them back without re-ticking anything.

4.2 PARAMS — choosing the axes of a tile

PARAMS sets the x channel and the y channels of the active tile, and nothing else; the legend, grid, log scale and marker settings that once lived here moved to **STYLE**. The x combo and the y list both display the channel's label — the name with its unit appended, as in `ALTITUDE [ft]`, when the file's dialect carries one — but both carry the bare name as their data, so a unit can never be mistaken for part of a name.

The filter box matches against the channel *name*, uppercased. If the text contains `*` or `?` it is treated as a wildcard pattern; otherwise it is a substring test. Non-matching rows are hidden rather than removed, so the ticks of channels you have filtered out of sight are still in force. Ticked channels become the tile's y list in the order they appear in the list, not the order you ticked them.

Right-clicking a channel offers **Create alias for NAME...**, **Use as X axis** and **Copy name**. The alias dialog is pre-filled with a readable guess made by replacing underscores with spaces and capitalising words.

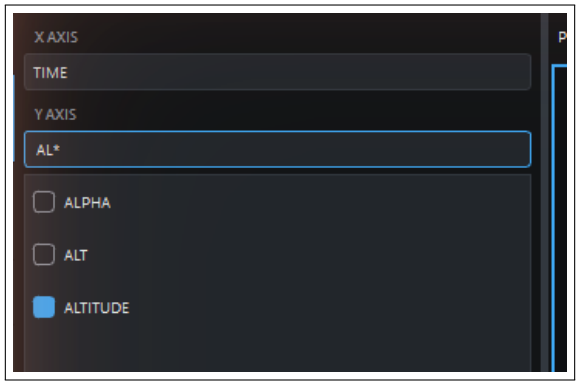


Figure 4.3: The **PARAMS** panel with the filter `AL*` typed, which has reduced thirty-nine channels to the three whose names match.

Refreshing the panel *displays* a sensible x channel — the first whose name starts with `TIME`, otherwise the first channel — but does not write it to the tile. The default is committed once per redraw, by `AppState.apply_parameter_defaults()`, and only for tiles that already have a y or z channel. Blank tiles are left blank.

4.3 COMPUTE — channels the file does not contain



Figure 4.4: Left: a name and an expression, with the validator's verdict beneath. Right: the list of defined parameters, each shown as `name = source`.

A computed parameter is a name and an expression. Type both and press **Create**; the expression is validated on every keystroke, and the line beneath the field reads **Valid** or carries the core's parse error. Validation evaluates nothing — it parses and resolves identifiers only — so it is cheap enough to run while you type.

Once created, the parameter behaves exactly like a channel read from the file. It appears in **PARAMS**, in **ISO**, in the **INSPECT** tables, in the CSV export, and it can be referenced by other formulas. **Edit selected** loads a definition back into the fields; the name field is disabled while editing, because renaming would orphan every expression that referred to it. Applying an edit re-evaluates the parameter everywhere. **Remove selected** removes it, and also removes everything that depended on it, saying which: *Removed MACH_SQ, and ENERGY which depended on it.*

The **Help** button opens the complete syntax reference, which is also reproduced in Chapter 9.

4.4 ALIAS — a second name for a channel

Choose a channel, type a display name, press **Add alias**. The new name appears everywhere a channel name can appear and resolves to the same data. Aliases are resolved in the core, chains are



Figure 4.5: `CLIMB_RATE`, defined as `deriv(ALTITUDE, TIME)`, plotted for all twenty-five runs. It was never in the file: the core evaluates it per run, and the tile cannot tell the difference between it and a channel that was.

followed, and a cycle such as `A -> B -> A` is broken rather than hung. The defined list shows each alias with the name it resolves to, and **Remove selected** deletes one.

> An alias is written into the workspace and restored with it. If the channel behind it is not in the files the workspace reopens with, the alias is *kept* rather than dropped — it costs nothing until it resolves, and the file it belongs to may be opened next — and the load reports it so that an axis plotting nothing has an explanation.

4.5 ISO — filtering runs by a value they hold constant

An *ISO* parameter is one whose value is constant within a run but differs between runs: the initial mass of a sweep, a Mach number, a configuration index. The **ISO** tab filters the runs by those values, and several ISO parameters can filter at once, combining with a logical AND so that each one narrows the set further.

Choose a parameter in the **Filter by** combo, which lists only parameters not already filtered on, and press **Add**. The parameter appears as a checkable node whose badge summarises the selection — *all 75*, *none of 75* or *2/3* — followed by *grouped* when it is grouped, *not in the loaded files* when no open file carries it, or *N outside the groups* when some values fall outside every interval. Its children are the individual values, or the interval names once grouped, each separately checkable. The node's own checkbox is the filter's active flag; ticking any value also activates the filter, since ticking a value is a statement about what to show and is meaningless while the filter is off.

Four buttons act on the selected parameter: **Group...** opens the interval editor, **All** and **None** tick or untick every value, and **Remove** stops filtering by it. The same operations, plus **Ungroup**, are on the context menu. Ungrouping keeps the intervals in state, so re-grouping returns to what you had set up.



Figure 4.6: The **Formula reference** dialog. Operators, element-wise functions, the three logarithms, series functions, reductions, constants, the lookup form and a dozen worked examples. The logarithm section is reproduced in §9.3.1.

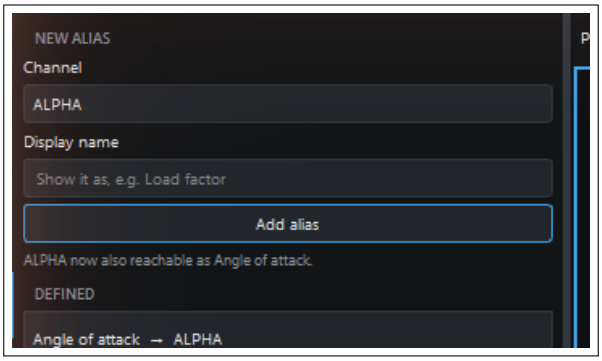


Figure 4.7: An alias being created. `ALPHA` keeps its name and gains a second one; both resolve to the same samples.

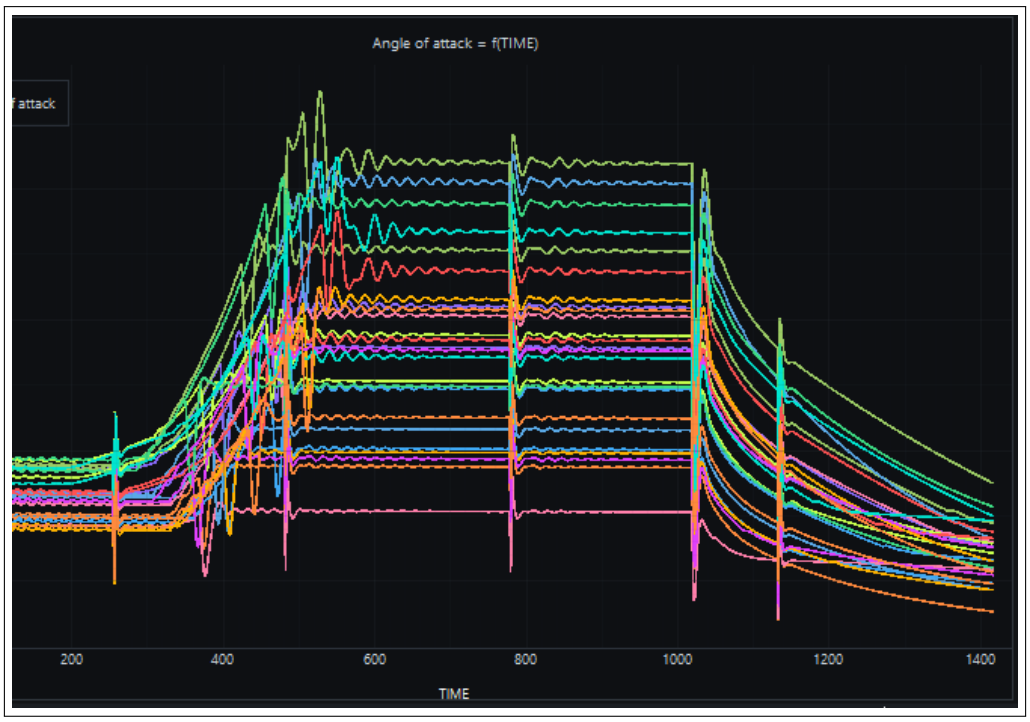


Figure 4.8: A tile plotted through an alias. The axis label, the legend row and the tile title all read *Angle of attack*; the samples are `ALPHA`.

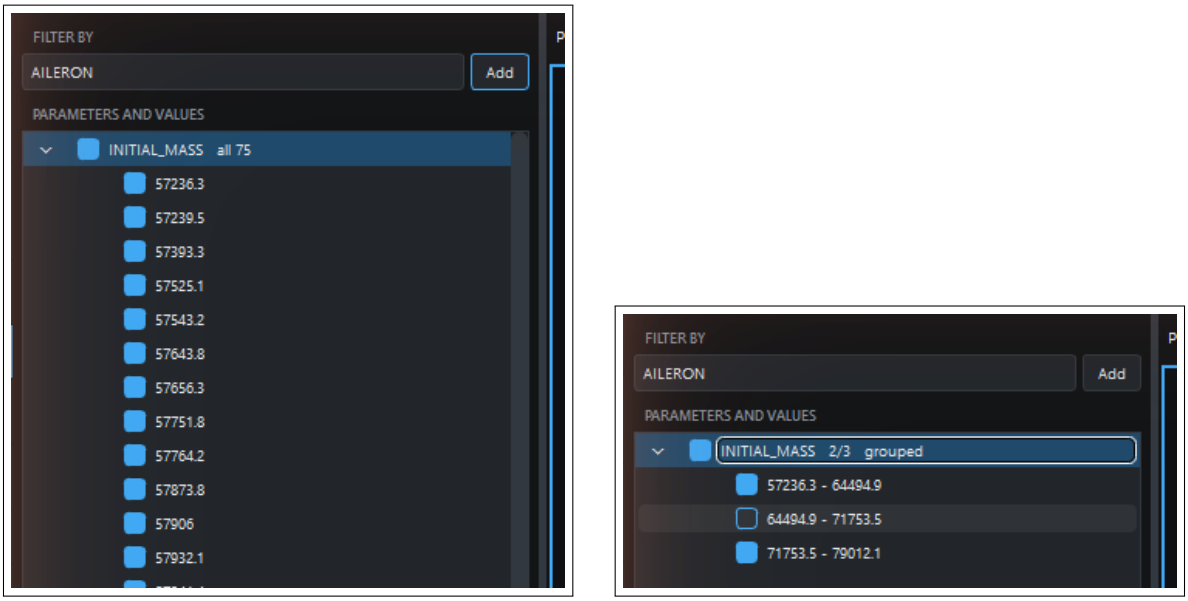


Figure 4.9: Left: `INITIAL_MASS` added as a filter, ungrouped, showing all seventy-five distinct values with the badge *all 75*. Right: the same parameter after grouping into three intervals, with the middle one unticked — the badge now reads *2/3*.

4.5.1 Grouping values into intervals

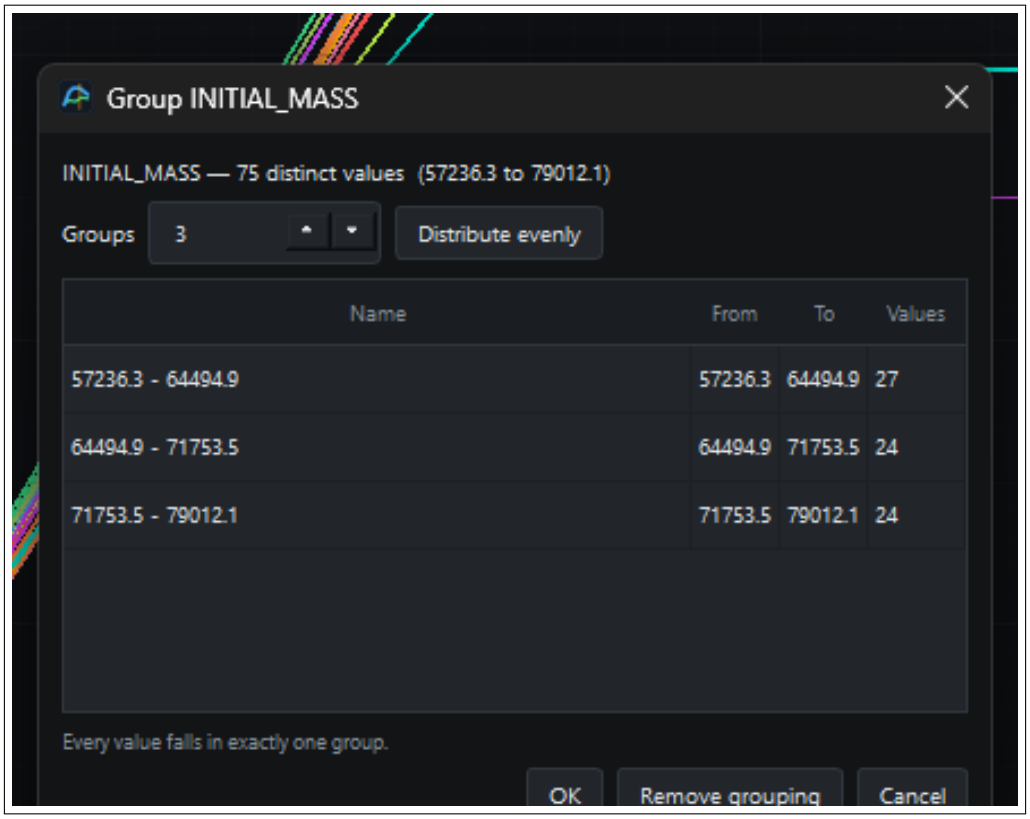


Figure 4.10: The grouping dialog. Seventy-five distinct masses between 57 236.3 and 79 012.1 have been split into three intervals; the *Values* column counts what falls in each, and the note underneath confirms that every value falls in exactly one group.

The header states how many distinct values there are and the range they span. The **Groups** spin box is bounded above by the number of values — there can never be more groups than values — and **Distribute evenly** splits the observed range into equal intervals, discarding boundaries edited by hand. The table is editable in its *Name*, *From* and *To* columns; *Values* is read-only.

The editing model is the reason overlapping intervals are impossible rather than merely rejected. N groups are stored as $N + 1$ sorted edges, so the *From* of one row and the *To* of the row above are the same number; editing either moves that single edge, clamped between its neighbours. Interior gaps and overlaps are therefore unrepresentable. The two outer edges may still be moved inward, which is how an outlier is deliberately excluded, and the note beneath the table then reports how many values fall outside every group. A comma is accepted as a decimal point, and a cell that cannot be parsed silently restores its previous value. Changing the group count clears hand-typed names and re-splits evenly; names you typed yourself are never regenerated.

Bounds are inclusive at both ends and the first claiming group wins, so a value sitting exactly on a shared edge belongs to the earlier interval.

4.5.2 Two constraints, composed

The decisive property of this panel is what it does *not* do: applying a filter never writes into the **DATA** selection. A run is drawn if and only if

$$\text{ticked in } \textbf{DATA} \;\wedge\; \text{admitted by every active } \textbf{ISO} \text{ filter.}$$

Ticking a run whose ISO value is excluded therefore puts it on no tile, and changing an ISO filter never re-ticks the **DATA** tab. The panel's summary line says how many of the **DATA** tab's runs are being held back, and the status bar repeats it.

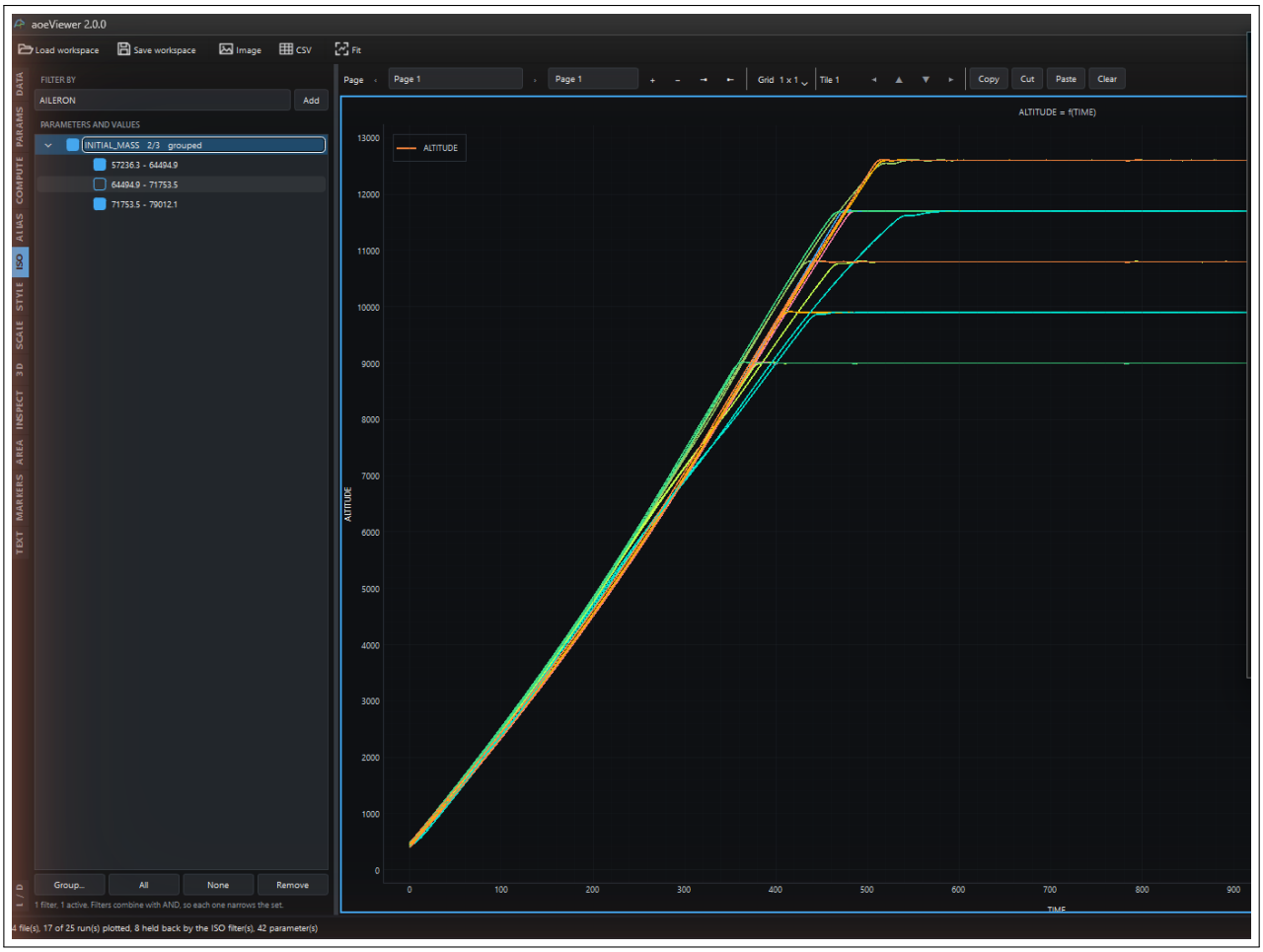


Figure 4.11: The middle mass interval has been unticked — the node now reads *2/3*. Eight of the twenty-five ticked runs are held back and their curves are gone; the **DATA** selection is untouched, and re-ticking the interval restores them. The status bar states both numbers. The window is shown cropped at the right.

An earlier version wrote the filter result *into* the run selection. Applying a mass filter then silently re-ticked the **DATA** tab, and re-ticking a run in **DATA** appeared to widen the filter: two controls, one piece of state, each overwriting the other's answer. The intersection is now computed only where curves are actually drawn, and memoised, because every tile asks the same question on every rebuild.

Two details of the admission rule are worth stating. A run whose value for the parameter is not a number is never admitted — it has no value for that parameter, so no interval can claim it. And for an ungrouped filter, a value absent from the tick map counts as *on*, so a value that appears only after a new file is opened is shown rather than silently hidden.

4.5.3 What a workspace keeps, and what it does when the data has moved on

The filter *definitions* are saved: which parameters are filtered on, their intervals, and which values are ticked. What is not saved is the *answer* — which runs those filters admit describes the files the last workspace had open, and is rebuilt once the new ones are known.

A workspace outlives the run it was made from. The solver is re-run with fewer channels, a file is replaced, a parameter is renamed; none of that may raise, and none of it may quietly empty the plot. Three cases are distinguished, and the load reports each of them in one dialog titled **Workspace loaded with differences**.

Situation	What happens	Shown as
No open file carries the parameter	The filter constrains nothing: it cannot speak about a channel that is not there.	*not in the loaded files*
The parameter is there, but no run falls in a ticked interval	The filter genuinely admits nothing, and the plot is correctly empty.	*no run satisfies it*
The column is there but a run's value is not a number	That run alone is excluded — the column exists and this run has no value for it.	greyed in **DATA**

Table 4.1: How a restored filter behaves against data that no longer matches it.

> The first row of that table used to read the other way: a filter on a parameter a file did not carry excluded every one of that file's runs. That is defensible read on its own and wrong in every case it actually arises. It made a filter on a parameter only *some* files carry hide every run of the others, and — once filters became part of a saved workspace — it made a workspace whose ISO parameter had since disappeared open to an empty plot with nothing at all to explain it.

Entries that cannot be read at all are skipped individually rather than failing the load: a value that is not a number, an interval with non-numeric bounds, an entry that is not an object. A reversed interval is normalised rather than refused, and a grouped filter whose intervals did not survive is demoted to ungrouped rather than left in a state the tab would have to invent intervals for.

Very long value lists are truncated in the tree at four hundred children, with a non-checkable row reading *… and N more, still filtered on*. The ticks for those values live in the state, so the filter still applies to them; only the rows are omitted, because laying out several thousand tree items is slow enough to be felt.

4.6 STYLE — colour, line, marker and legend

The panel opens with four per-tile switches — **Legend**, **Grid**, **Log Y** and **Points** — whose defaults are on, on, off and off. **Points** draws a circular marker on every curve of the tile that has no marker of its own; a per-curve marker overrides it.

4.6.1 What decides a curve's colour

Three modes, chosen by radio button, decide which entity owns a style.

Choosing **ISO** reveals an **ISO parameter** combo. If the tile has no ISO parameter yet, the combo displays the default without writing it, and selecting the radio commits that choice immediately — so that the *Curves* table below is built from the same parameter the plot will use. Getting this wrong was a real defect: the table was built from the old choice and showed the wrong rows until something else forced a refresh.

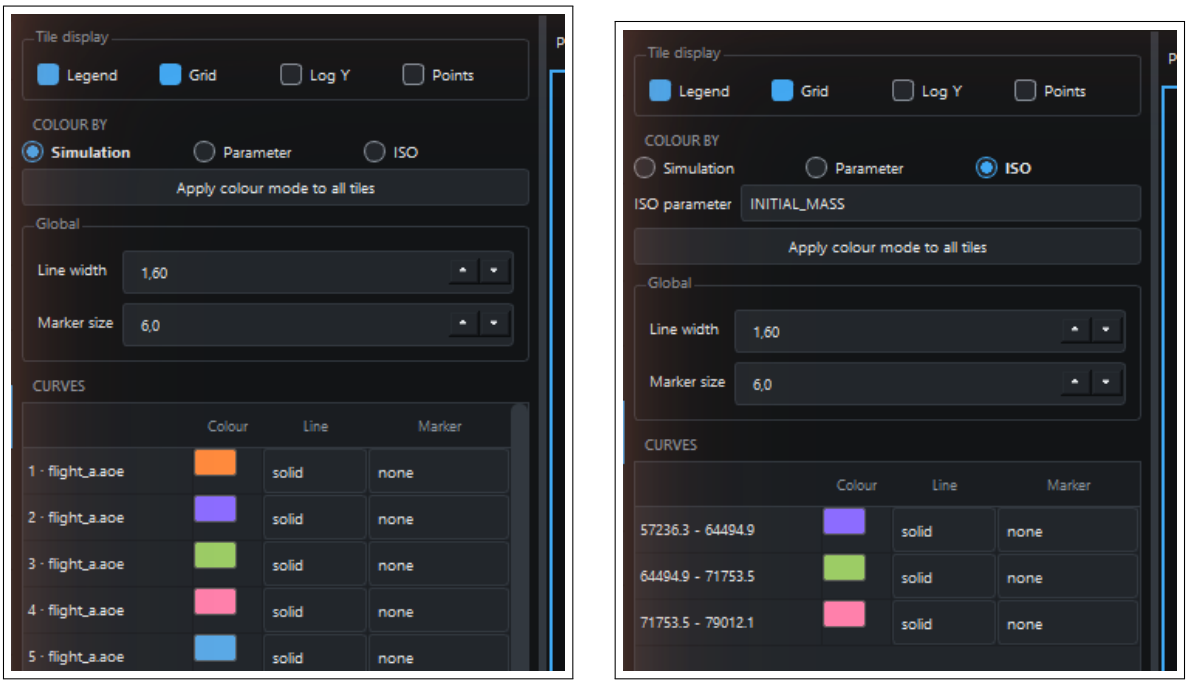


Figure 4.12: Left: colouring by simulation, so the *Curves* table has one row per plotted run. Right: colouring by ISO, so it has one row per mass interval — three rows governing twenty-five curves.

Colour by	One style per	Key
Simulation	plotted run	`sim::<run key>`
Parameter	y channel of the tile	`param::<name>`
ISO	value or interval of the chosen ISO parameter	`iso::<param>::<label>`

Table 4.2: The three colouring modes and the key namespaces that keep them from colliding.

A colour, once allocated, is kept for the session. The palette has twelve entries and the registry hands them out in order of first use, so the **first three curves on a tile are always red, then blue, then dark green**, whatever the colouring mode — allocation depends on the order keys are first seen, not on what they describe. Three curves is the common case, and a set of figures whose three colours mean the same three things throughout is easier to read than one whose first colour depends on how many curves happened to be on the tile. There is deliberately no cyan near the front: cyan and a light green are the same colour once a line is one pixel wide. The remaining entries are ordered for separation. Loading another file therefore never recolours what is already on screen. **Re-assign palette** deliberately forgets the current assignments so the next redraw allocates afresh, and **Reset widths** drops per-curve widths so all follow the global setting. Two global spin boxes set the line width, from 0.1 to 10.0 with a default of 1.6, and the marker size, from 1.0 to 20.0 with a default of 6.0.

The *Curves* table has one row per styleable entity in the current mode, with a colour swatch that opens a colour dialog, a dash combo over *solid*, *dash*, *dot* and *dashdot*, and a marker combo over eleven shapes from *none* to *hexagon*. When nothing is plotted it shows a single unselectable row reading *Nothing plotted yet*.

4.6.2 The legend

A legend row is assembled from up to four parts, each switched separately: **Parameter** (on by default), **ISO**, **File** and **Simulation**, joined by vertical bars. The dash pattern and marker are never spelled out in words, because the legend already draws a sample of the curve.

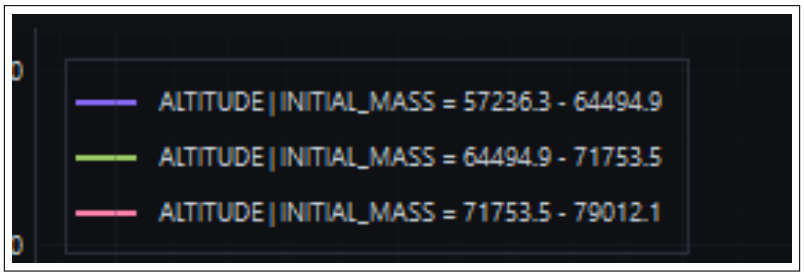


Figure 4.13: Twenty-five curves, three legend rows. With **File** and **Simulation** off, identical rows are merged — which is precisely what turns a mass sweep into one row per mass interval.

That merging is the point of the switches: with file and run names off, twenty-five runs sharing three ISO intervals produce three rows. Beyond sixty rows the legend stops adding them and appends a pen-less entry reading *... and N more*, so the truncation is stated rather than silent; pyqtgraph lays out two graphics items per row, and a thousand-run study froze the application here before the cap existed.

The **Corner** combo snaps the legend to any of the four corners. Dragging the legend on the plot puts it anywhere, switches the corner to *custom* and keeps the position, including in a saved workspace. **Apply legend to all tiles** copies this tile's whole legend configuration — whether it is shown, its corner or dragged position, and which parts make up a row — onto every tile of every page, and reports how many tiles changed or says that every tile already had it.

4.7 SCALE — axis ranges and cross-tile constraints

While an axis is unlocked, its two boxes *follow* the plot as you pan and zoom, so ticking **Lock X range** or **Lock Y range** is a one-click way to freeze exactly what you are looking at. Locked, the boxes are the range: they are enabled, and typing in one moves the frame. The boxes span $\pm 10^{12}$ with six decimals and do not commit per keystroke — a value takes effect on Enter or when the field loses focus. A minimum typed above its maximum is normalised rather than allowed to blank the plot.

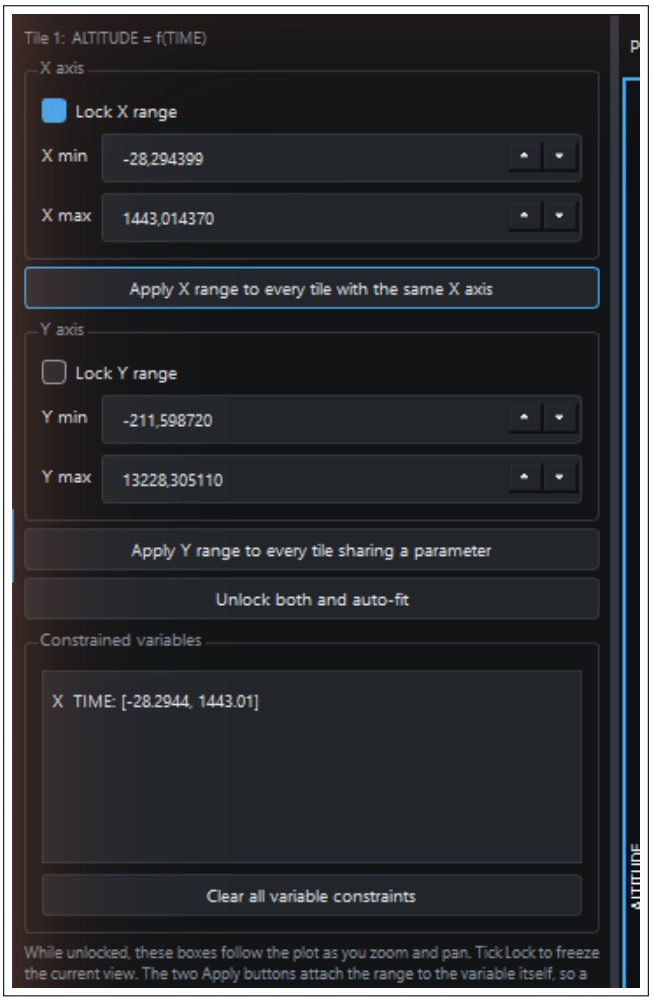


Figure 4.14: The **SCALE** panel with the x axis locked. The four spin boxes are enabled only while their axis is locked; the *Constrained variables* list shows that `TIME` now carries a range attached to the variable itself.

A locked range is a hard bound, not a starting point: the view's limits are set and per-axis mouse interaction disabled, so no drag or wheel can leave it. A degenerate range is refused, because a view frozen at zero width looks exactly like a crash.

The two **Apply** buttons attach a range to the *variable* rather than to the tile. **Apply X range to every tile with the same X axis** gives every plot against that x channel, on every page, the same horizontal scale, so they can be compared directly. **Apply Y range to every tile sharing a parameter** does the same for any tile showing any of this tile's y channels. Because these constraints are re-applied on every redraw, they also govern tiles that acquire the parameter *later* — which is the whole point, and which the first implementation got wrong by walking only the tiles that existed when the button was pressed.

4.7.1 When two constraints meet on one axis

A tile has one y axis and may draw several channels on it, so it can be asked for two different vertical windows at once. Consider three tiles: the first draws `ALTITUDE`, the second draws `ALTITUDE` and `MACH`, the third draws `MACH`. Constrain `ALTITUDE` from the first tile and `MACH` from the third, and the second tile now carries two incompatible demands.

No arithmetic satisfies both, so none is invented. Parameters that ask for the same window are applied normally. Parameters that disagree leave the tile exactly as you had it — free, and therefore auto-fitting to show all of its data — and the **SCALE** tab says so, naming the parameters and their ranges, with the offending entries marked in the *Constrained variables* list. A button, **Use the range that covers both**, then offers the compromise as something you choose rather than something you are given; it locks *this tile*, not the variables, so the tiles whose scales the constraints were set to fix keep them.

> Taking the smallest window containing both is the obvious answer, and it is what the program used to do. It is wrong twice over. A union is a superset of both ranges and therefore honours neither, which defeats the entire purpose of the feature: the point of constraining a variable is that two tiles showing it can be compared directly, and under a union they cannot. And when the two ranges differ in magnitude the union destroys the smaller channel — `ALTITUDE` at $[0, 13000]$ unioned with `MACH` at $[0, 1]$ leaves Mach occupying 0.008 % of the axis height, a flat line along the bottom edge, with nothing on screen to explain it. Which parameter survived depended on its magnitude rather than on any decision the user had made.

The x axis cannot fail this way. A tile has exactly one x channel, so the lookup returns at most one range however many x constraints are declared; the conflict is not representable. The same channel may carry one window as an x axis and a different one as a y axis, which is correct — they are different axes — and the *Constrained variables* list labels every entry with the axis it belongs to.

Three-dimensional tiles are skipped entirely. Their frame is the projection's own centre and zoom, and the two-dimensional ranges describe data extents that mean nothing to a projection; locking them did no visible harm but left a tile mysteriously locked if it was ever switched back to two dimensions.

The *Constrained variables* list shows every such constraint, and **Clear all variable constraints** removes them while leaving per-tile locks alone. **Unlock both and auto-fit** releases this tile and also pops its parameters out of the variable-wide constraints — without which the next redraw would simply re-lock it.

A tile is fitted once, when it first acquires data. After that the frame moves only on **Fit**, on a double-click, or when **SCALE** changes something. An ordinary redraw — restyling a curve, ticking a run, toggling the grid — never moves the frame. Locking x on a tile whose y is free is the one exception: the y axis is re-fitted *inside* the new x window, computed by a single pass in the core that understands a non-monotonic x, because otherwise a windowed manoeuvre sits as a flat line at the bottom of the tile.

4.8 3D — three channels against each other

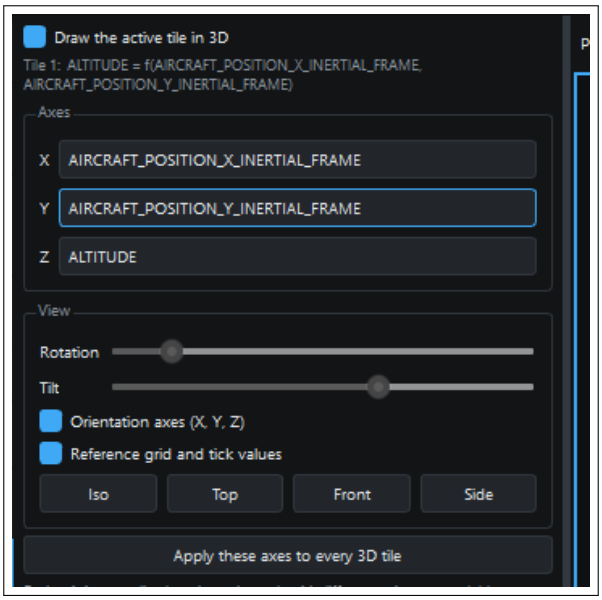


Figure 4.15: The **3D** panel. Every axis and view control is disabled until **Draw the active tile in 3D** is ticked.

Ticking **Draw the active tile in 3D** turns the active tile into an orthographic projection of three channels, filling in plausible axes if the tile has none. The three combos set x, y and z; a 3D tile uses exactly one y channel. **Rotation** sweeps the azimuth from 0 to 359 degrees and **Tilt** the elevation from -89 to $+89$, clamped rather than wrapped because past straight up or down the projection turns inside out. Four presets — **Iso**, **Top**, **Front**, **Side** — set the pair directly, and **Apply these axes to every 3D tile** copies axes and view onto every other tile already in 3D.

Two switches control the depth cues. **Orientation axes (X, Y, Z)** draws the fixed-size triad in the corner, coloured by the universal convention — x red, y green, z blue — and hovering it names the channel on each axis and its range. **Reference grid and tick values** draws the back-wall grid and its numbers; without them the projection has no depth cue and no numeric scale, but on a very dense trajectory they can be turned off.

Each channel is normalised independently onto $[-0.5, 0.5]$, so three channels with unrelated units stay readable together; a channel that never moves collapses to the middle of the cube, which is the honest picture of a constant. In **Pan** mode a drag turns the object, middle-drag or Shift+drag slides it and right-drag or Ctrl+drag zooms; in **Zoom** mode the wheel zooms. Double-click or Ctrl+0 frames everything again.

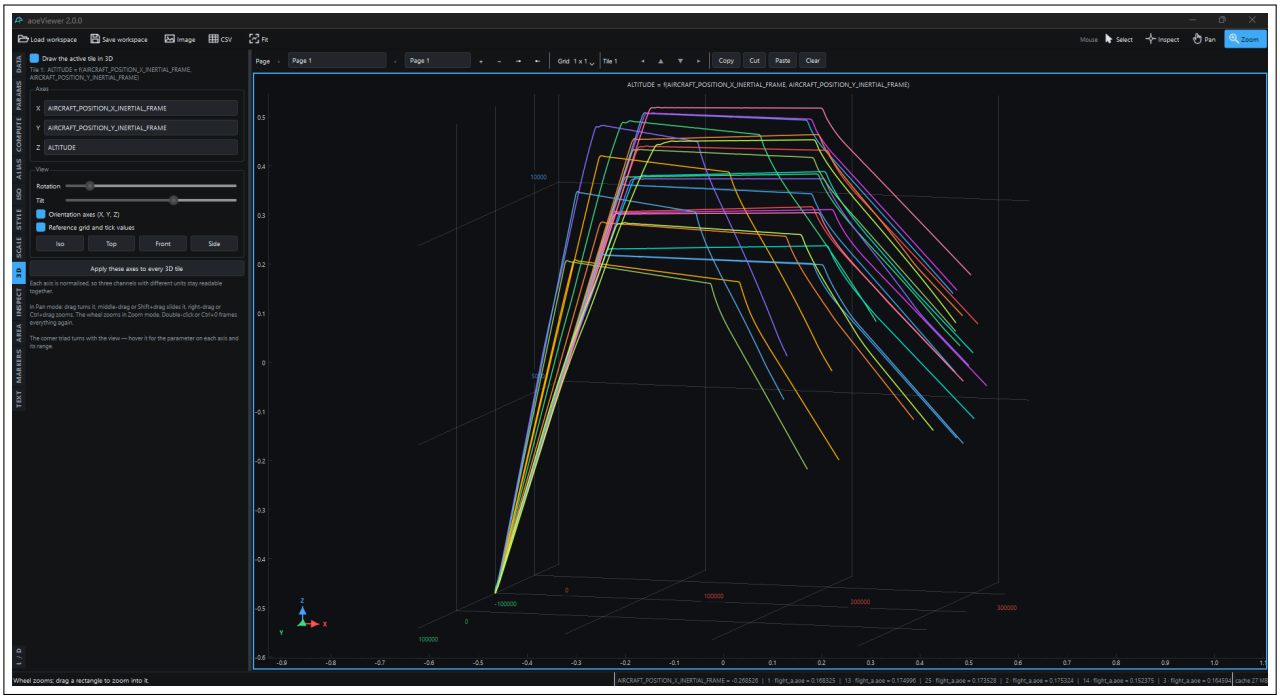


Figure 4.16: Twenty-five trajectories: ground track east against north, with altitude as z. The grid is drawn on the three faces of the data box that lie *behind* the trajectory from the current viewpoint, and swaps faces as the object turns. The triad in the corner keeps a constant size however far you zoom.

A vertical line is not a plane, and the indices a marker dot would use refer to the thinned, projected arrays rather than to samples. Annotations are therefore restricted to two-dimensional tiles.

4.9 INSPECT — reading values off a curve

Switch the mouse to **Inspect** and click a curve. The header names the file, the run and the x value at the click; the *At a point* tab then lists every channel of that run at that sample, with its unit, printed to six significant figures and with a dash where a value is not a number. **Copy to clipboard** writes the header and one tab-separated line per visible row — visible meaning after the channel filter, which accepts the same wildcards as **PARAMS**.

The nearest curve is found in *pixels*, not in data units. The two axes carry unrelated scales, so a distance measured in data space would always pick the channel with the larger numbers.

The *Statistics* tab gives minimum, maximum, mean, standard deviation and sample count for every combination of plotted run and y channel of the active tile. By default it reports over the visible x window and follows pan and zoom; **Whole channel instead of the visible range** widens it. Its header states the scope exactly, including the mixed case that arises when some runs have an x channel shorter than their y channel and therefore cannot be windowed at all. Both tables copy to the clipboard tab-separated.

Dragging in **Inspect** mode measures a gradient instead. A dashed line follows the pointer with the running figure beside it, and on release the panel reports the slope, the two deltas, both endpoints and the inverse. A purely vertical drag reports the change in y over no change in x rather than an infinity. A drag shorter than six pixels is treated as a click, so a slightly unsteady hand still inspects a point.

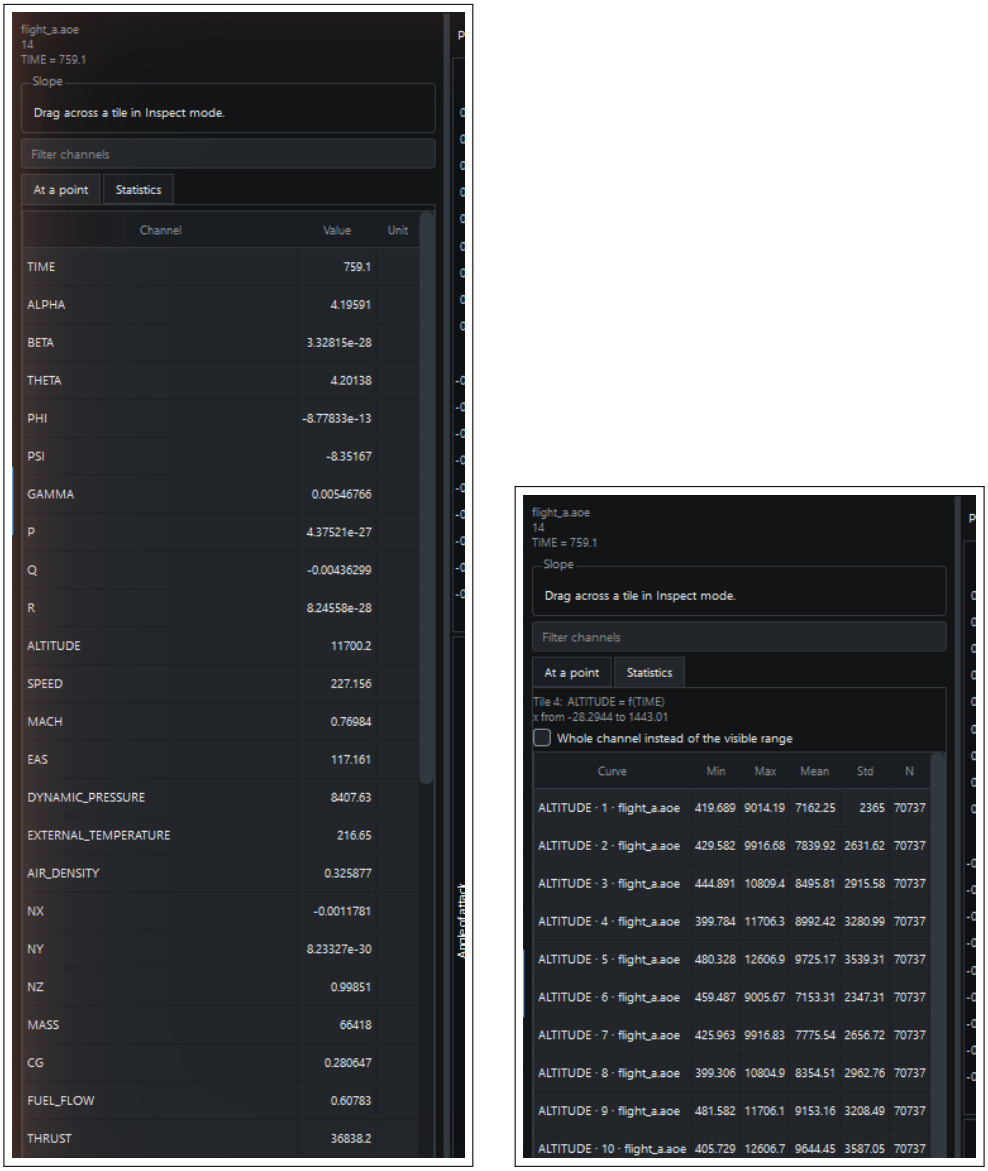


Figure 4.17: Left: every channel of one run at one instant, read after clicking a curve in **Inspect** mode. Right: per-curve statistics over the visible x window.

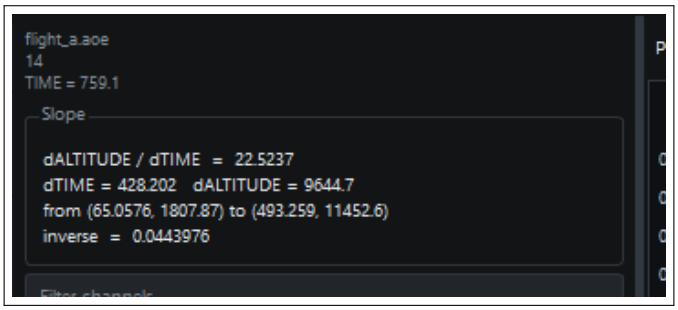


Figure 4.18: A slope measured by dragging across the climb. The gradient, both deltas, the two endpoints and the reciprocal are all reported; the reciprocal is there because on a time axis it is the quantity you usually want next.

4.10 AREA — shaded bands

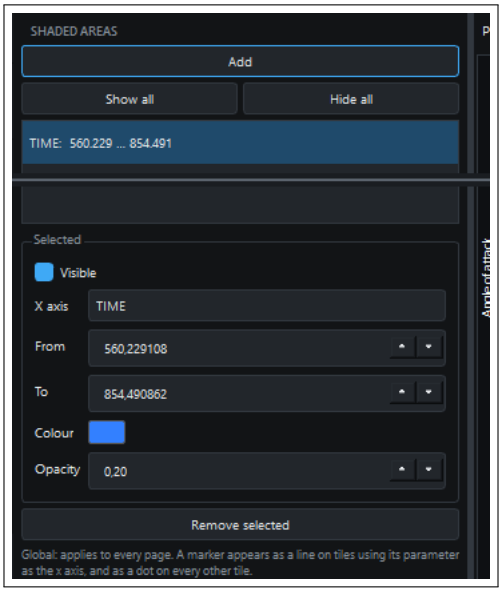


Figure 4.19: The **AREA** panel. Bands are global: they appear on every page, on every tile whose axes can express them.

Add places a band across the middle of what the active tile currently shows — from 40% to 60% of the visible x span — using that tile's x channel. The editor beneath sets visibility, the x channel, the two bounds, the colour and the opacity, which runs from 0.02 to 1.0 with a default of 0.20. The bounds are normalised on commit, and the two buttons **Show all** and **Hide all** act on every band at once.

In **Select** mode the edges of a band can be dragged directly on the plot, and the panel follows. There is deliberately no caption field: a caption typed here rebuilt the tile on every keystroke, deleting the very item being edited. Name a band with a **TEXT** note placed over it.

4.11 MARKERS — an instant, drawn two ways

A marker is a value of one channel — `TIME = 707.35`, say. **Add** places one at the midpoint of the active tile's x window. The editor sets visibility, the channel, the position, the colour and the dot size, which runs from 2.0 to 30.0 with a default of 9.0.

The dual rendering is the feature. On a tile whose x axis *is* the marker's channel the marker is a dashed vertical line, draggable in **Select** mode. On any other tile it becomes a dot on each curve, at the sample where the condition holds — which says something a vertical stripe could not.

A marker that falls outside a run's range is refused rather than clamped to the last sample, because a clamped dot would silently claim the condition had been met. The band's equivalent rendering redraws the matching stretch of every curve in the band's colour, with separate passes through the band left unjoined.

4.12 TEXT — captions on a tile

Captions belong to the active tile — the deliberate opposite of markers and areas — and are positioned as a fraction of it, from 0.0 to 1.0 in each direction with `y = 0` at the top. **Add text** appends a note reading *Note*, staggered downward from any notes already there. The editor sets visibility, the text (several lines are fine), the position, the point size from 4 to 72, bold, and the



Figure 4.20: One band, three tiles. Every tile whose x axis is `TIME` shows it as a shaded region; the three-dimensional tile shows nothing, and the band is drawn *in front of* the curves, because a band underneath them lost its edges wherever a curve crossed one.

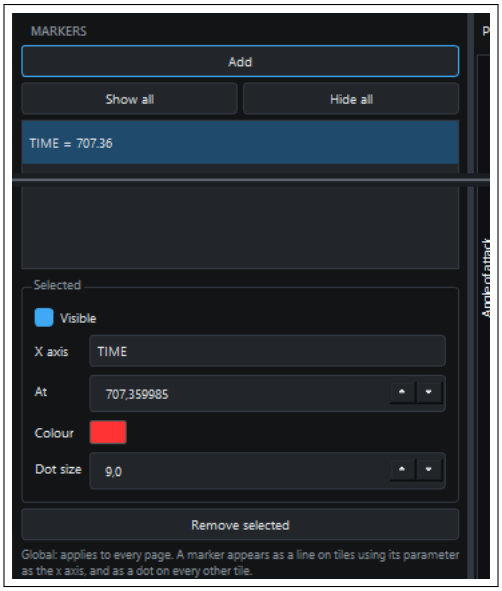


Figure 4.21: The **MARKERS** panel. **Dot size** governs the second of the two renderings described below.



Figure 4.22: The marker as a dashed vertical line, on the three tiles whose x axis is the marker's own channel.

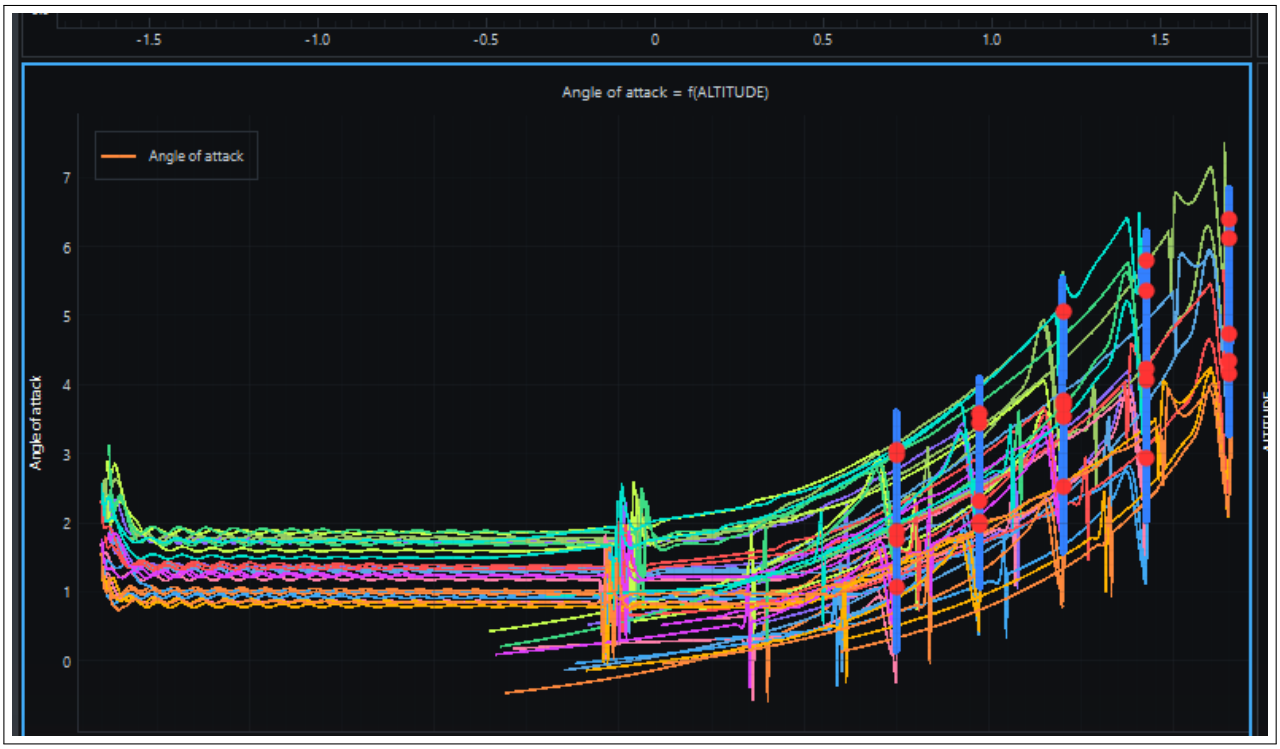


Figure 4.23: Angle of attack against altitude, with the same marker and the same band as the previous figures. The marker is now a red dot on each curve at the instant `TIME = 707.35`; the band is the thick blue stretch of each curve during `TIME` 580–854. On a trajectory this answers a question a vertical stripe cannot: *where was the aircraft during those ten seconds?*

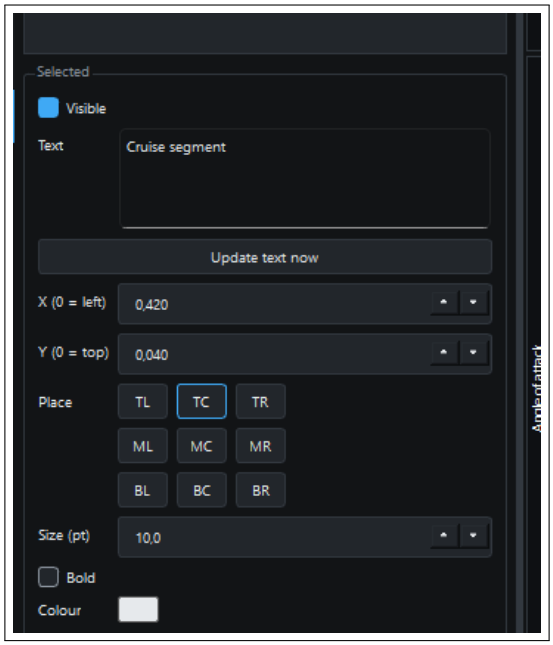


Figure 4.24: The **TEXT** editor. The nine **Place** buttons are corner and edge presets; **X** and **Y** are fractions of the tile, so a caption stays put through pan and zoom.

colour, which starts as the current theme's text colour. **Copy to all tiles** deep-copies this tile's captions onto every visible tile of every page.

Typing does not commit on every keystroke: a 400 ms idle timer fires after you stop, or the caption commits when the field loses focus, or immediately on **Update text now**. Committing per keystroke used to flicker the tile and steal the caret. Notes can also be dragged directly on the plot in **Select** mode, and the spin boxes follow; clicking outside every note passes the click through, so empty space still selects the tile.



Figure 4.25: A caption placed at top centre of the altitude tile, over the shaded band it names.

CHAPTER 5

Pages, grids and tiles

5.1 The page and tile bar

The bar immediately above the grid governs layout. On the left, a combo switches page, flanked by previous and next buttons, and a name field renames the current page on Enter or when it loses focus. Four small buttons add a page, remove the current one, and move it earlier or later in the order. Removing the last remaining page is refused.

In the middle, the **Grid** button drops a five-by-five matrix in the manner of a word processor's table picker. Sweeping the pointer highlights the layout and the caption underneath reads, for example, *2 x 3 (6 tiles)*; releasing chooses it.



Figure 5.1: The grid picker, mid-sweep. Cells whose layout would exceed twenty-five tiles are painted faint and cannot be chosen.

To its right, **Tile N** names the active tile and four arrows swap it with its neighbour to the left, above, below or right; each arrow is enabled only when that move is possible, and the active index follows the tile so the move is its own inverse. Finally **Copy**, **Cut**, **Paste** and **Clear** act on the active tile's whole configuration. Pasting installs an independent deep copy, so pasting twice gives two tiles that can then diverge. This clipboard lives in the process only and is not saved to a workspace.

5.2 How a page is modelled

Every page always carries twenty-five tile objects regardless of its grid, and the visible count is the product of rows and columns, capped at twenty-five. Shrinking a grid therefore *hides* tiles rather than destroying them, and they return unchanged when it grows again. Only the visible tiles are written to a workspace; loading pads the list back up to twenty-five.

The tile widgets themselves are recreated only when the shape of the grid changes. Constructing a pyqtgraph plot widget costs tens of milliseconds, so a fresh 5×5 page costs most of a second; switching between two pages of the same shape reuses the widgets and merely redraws them.

Field	Meaning	Default
`x_param`, `y_params`	the channels drawn; a 3D tile adds `z_param`	empty
`show_legend`	legend visible	on
`legend_position`	corner, or `custom` once dragged	`top-left`
`legend_x`, `legend_y`	dragged position as a fraction of the view	0.02, 0.02
`legend_show_param`	the channel name appears in a legend row	on
`legend_show_iso`	the ISO parameter and this run's value appear	off
`legend_show_file`	the file name appears	off
`legend_show_sim`	the run's own identifier appears	off
`lock_x`, `lock_y`	the axis is frozen at the stored range	off
`x_min`, `x_max`	horizontal range, in data units	0.0, 1.0
`y_min`, `y_max`	vertical range, in data units even in log mode	0.0, 1.0
`view_initialised`	this tile has been fitted once already	off
`log_y`	logarithmic y axis	off
`show_grid`	grid lines	on
`show_points`	fallback circular marker on every curve	off
`color_mode`	simulation, parameter or ISO	simulation
`color_iso_param`	the ISO parameter that colours the tile	empty
`plot3d`	draw as an orthographic projection	off
`azimuth`, `elevation`	view angles in degrees	45.0, 25.0
`show_3d_axes`, `show_3d_grid`	triad and back-wall grid	on, on
`view3d_cx`, `view3d_cy`, `view3d_half`	3D pan centre and zoom half-width	0.0, 0.0, 0.95
`texts`	the tile's captions	empty

Table 5.1: The complete state of one tile. All of it is saved in a workspace.

5.3 What a tile remembers

The tile's title is generated from this state and states plainly when something is missing: *empty tile*, *no parameter selected (x: TIME)*, *ALTITUDE (no x axis selected)*, or *3D tile — choose X, Y and Z*. With more than four y channels the title lists the first four and appends the remaining count.

Two 3D view fields deserve a note. The pan centre and zoom half-width are kept separately from the two-dimensional ranges, because those describe *data* ranges and mean nothing for a projection. The default half-width of 0.95 is chosen because the normalised cube's half-diagonal is $\sqrt{3}/2 \approx 0.866$, so 0.95 frames the whole box at any rotation with a small margin.

5.4 Robustness when the files change

When the set of open files changes, the application re-points anything that no longer resolves and reports what it did in the status bar — for example *page 2, tile 3: x axis VTAS -> TIME*. A missing z channel and a missing colour-by parameter are cleared; a missing *x* axis falls back to the default. A missing *y* channel is deliberately left named: it plots nothing today and comes back when the file is reopened, which is friendlier when a study is loaded one file at a time. With no files open at all, nothing is changed, so closing everything does not destroy a layout.

CHAPTER 6

Exporting, and saving your work

6.1 Image

Image writes the active page as a PNG, grabbing the plot grid widget exactly as displayed: every visible tile with its legend, markers, shaded areas and captions, at screen resolution, without the toolbar, rail or panels. The extension is appended if you omit it.

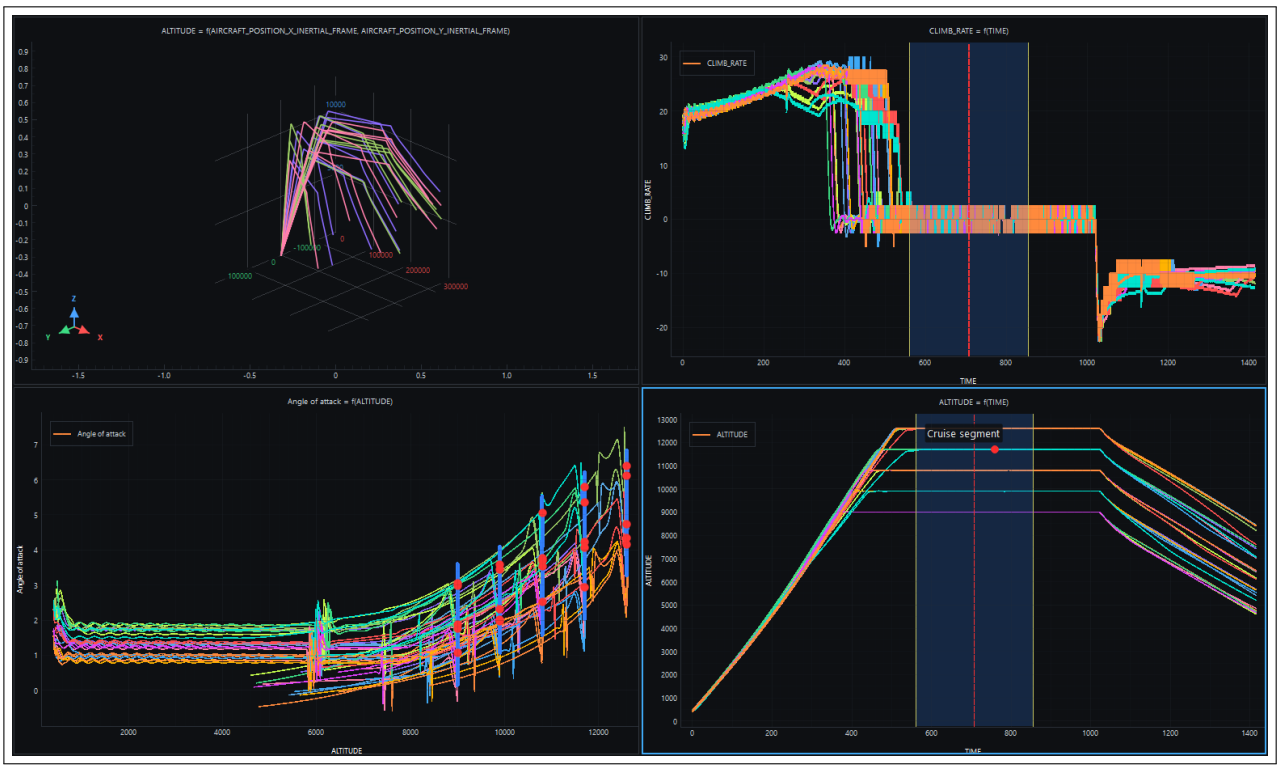


Figure 6.1: A page exported with **Image**. This figure is the file the button wrote, not a screenshot of the window — which is what makes it a useful way to get a figure out of the program and into a report.

6.2 CSV

CSV writes the active tile's curves at *full* resolution, on a worker thread. The header row is `file`, `run`, `parameter`, the name of the tile's x channel, and `value`; note that the fourth column header is the channel's name rather than a literal. There is then one row per sample per combination of plotted run and y channel, in the order the runs are plotted and then the order of the tile's y list, with both numbers written to nine significant figures. Runs whose x or y channel is empty are skipped, and the status bar reports the sample count written.

> The plot is decimated to roughly two points per horizontal pixel. The export is not: silently handing someone twelve hundred of their five hundred thousand points would be a trap. If the tile is empty or nothing is plotted, the button says so rather than writing an empty file.

6.3 Workspaces

A workspace is a single JSON file with the extension `.aoews`. It carries the list of open files by absolute path, every computed parameter, every alias, every ISO filter with its intervals and ticks, every page with its visible tiles and their complete state, the run selection, the whole style registry, the markers, the shaded areas, the variable-wide scale constraints, the theme and the mouse mode. What it deliberately does not carry is any *derived* answer — which runs the ISO filters admit describes the files that were open when it was written, and is recomputed against whatever is open when it is read.

6.3.1 Save, and Save as

There are two buttons, not one with a modifier, because which file is being written is the thing that gets got wrong. **Save workspace** (Ctrl+S) overwrites the current workspace, asking for a path only if there is not one yet. **Save workspace as...** (Ctrl+Shift+S) always asks, and then continues in the new file — leaving the one you came from exactly as it was on disk. That is how a variant is made.

Three places say which file you are in. The window title carries its name, with a leading asterisk while there are unsaved changes. The **Save** tooltip names the file it would overwrite in full, or says that there is no workspace file yet and it will ask. And the status line after a save repeats the path and what went with it — *Workspace saved to C:\study\sweep.aoews (including 2 formula(s), 1 alias(es), 3 ISO filter(s))*.

6.3.2 Loading against data that has moved on

Loading reads it, then does everything that touches the core on a worker thread: clears the session, opens each file, builds any missing sidecars, installs the aliases, and re-creates the computed parameters.

Aliases are installed before the formulas, since a formula may be written in terms of one. The formulas are then installed in as many passes as it takes, repeating until a pass installs nothing new. A single pass is not enough: a JSON object has no dependency order, so a formula defined in terms of another would be refused — correctly, as an unknown name — and silently dropped. Repeating settles any acyclic chain, and what is left over genuinely cannot be defined against this data.

Nothing here stops the workspace loading. Whatever resolves is in place, and one dialog titled **Workspace loaded with differences** then lists what could not be: files that could not be opened, formulas that could not be defined and why, aliases whose channel is absent (kept, in case it returns), ISO filters on parameters none of these files carry, and ISO filters that no run satisfies.

JSON was chosen over an ad-hoc delimited format because such a format breaks the moment a file path or a parameter name contains the delimiter, and cannot be extended without a version bump nobody implements. The loader is tolerant by design: it sets only the attributes it recognises and ignores the rest, which is how workspaces written when markers and bands still had a caption field continue to load.

> The active tile index, the tile clipboard, the memory limit, the window geometry, and the hidden tiles of a shrunken grid. A `"version"` key is written but nothing reads it, and no version-branching logic exists — the loader's tolerance of unknown keys is what stands in for it.

6.4 Background work, and why the interface freezes when it does

Anything that touches the core runs on a worker thread: opening files, building sidecars, loading a workspace, reading channels, reading every channel of a clicked run, and writing a CSV. This works

because `ctypes` releases the interpreter lock for the duration of every call, so a Python thread sitting inside a core function genuinely runs in parallel with the event loop — while the core is using all of its own worker threads inside that one call.

The governing constraint is that the core session is not internally synchronised, so only one thread may touch it at a time. Rather than take a lock, which would merely move the freeze from the paint loop to the click handler, the application disables the panels, the rail, the page bar and the grid while a task runs, and the tiles show a placeholder. Tasks queue rather than overlap, and for redraw work only the newest queued request is kept, since an older one is by then worthless. There is no cancellation: a running task cannot be interrupted.

Most redraws never reach a worker at all. Before dispatching, the application computes a signature of which channels each file must supply and which runs are plotted; when that matches what is already resident and no task is running, the grid is redrawn synchronously on the interface thread. Restyling a curve, toggling a grid or editing a limit is that kind of redraw, and routing it through a background task cost far more than the read it was meant to cover — starting a task disables and re-enables the panels, the rail and the whole grid, and every plot repaints on each of those transitions.

CHAPTER 7

The AOE file format

7.1 Overall shape

An AOE result file is plain ASCII text. There is no magic number, no preamble, no version field and no binary header; the only structure is a set of directive lines whose first non-blank character is an asterisk, with whitespace-separated decimal numbers between them.

```
*PARAMETRES
 'TIME'    'ALPHA'   'MACH'    'NZ'
*TABLEAU : SIM_001
 0.000  1.2340  0.7500  1.2500
 0.020  1.2350  0.7501  1.2499
*TABLEAU : SIM_002
 ...
*FIN
```

Listing 7.1: The canonical shape of an AOE file.

The parser recognises exactly two keywords, `*PARAMETRES` and `*TABLEAU` — note the French spelling of the first, without accent. A line is treated as a directive if and only if its first non-blank character is an asterisk; the keyword is then located by substring search within that line, which is why a form such as `*TABLEAU NUM SIMU: 1.2.0` is recognised as readily as `*TABLEAU : SIM_001`. Any other asterisk directive — `*FIN`, `*ENTITY`, a comment line — closes the open block and ends header reading. Nothing is required to terminate the file: `*FIN` is not special-cased, it is simply one more directive that closes the last block.

> Because nothing identifies the format, the only test of whether a file is an AOE file is whether parsing it yields at least one parameter. A file that maps successfully but contains no usable `*PARAMETRES` block is rejected; a file with headers but no `*TABLEAU` at all opens successfully with zero runs, because nothing rejects that case. Opening also fails if the file cannot be mapped, which deliberately includes a zero-length file.

7.2 Two header dialects

The `*PARAMETRES` block comes in two forms, and the parser chooses per line rather than per file.

The **packed** dialect puts several single-quoted names on one line and carries no units. Every non-empty quoted token on the line becomes a parameter, and its unit is the empty string.

```
*PARAMETRES
 'TIME'    'ALPHA'   'MACH'    'NZ'
```

The **indexed** dialect puts one parameter per line, prefixed by an ordinal and suffixed by a unit. The first quoted token is the name, the second — if present — is the unit, and anything further along the line is ignored.

```
*PARAMETRES
      1 'TIME'   's'
      2 'PARAMETER001'    ''
      3 'PARAMETER002'    'deg'
```

The test that distinguishes them is deliberately strict: digits, then only blanks, then a quote. A packed line begins with the quote itself and so can never match. The strictness matters more than it looks. Treating every quoted token on an indexed line as a name turns a 585-parameter file into 1156 parameters, puts `deg`, `ft` and `s` into the parameter list, and shifts every column index — so that asking for `TIME` silently returns another channel's samples. Correspondingly, quote extraction deliberately emits empty tokens, because the indexed dialect writes an absent unit as `''` and dropping it would shift the following line's unit onto the wrong parameter.

The header section is opened by the first `*PARAMETRES` encountered; a second block is ignored, which is what allows these files to repeat the header before every run. It is closed by whichever comes first: a line that yields no names once names have started, a blank line once names have started, or any asterisk directive.

7.3 How runs are delimited

A `*TABLEAU` line opens a run. Its identifier is everything after the first colon on the line, whitespace-trimmed; if there is no colon, or nothing after it, the identifier becomes `SIM_` followed by the run's one-based ordinal, without zero padding. The run's data begins at the start of the line following the marker, and its end offset is provisionally the end of the file and is rewritten the moment the next directive is met. That design — recording the end as the scan goes — is why the later parse tasks need neither a search nor a per-line terminator test.

Each run also carries a unique key of the form `flight_a.aoe::SIM_001::0`: the file name, the identifier from the file, and the ordinal. It is deliberately not derived from the display name, so renaming a run in the interface does not change its identity and therefore does not lose its colour.

7.4 Records, rows and wrapped lines

A logical record holds one value per header. Fortran writers wrap a record across several physical lines when it exceeds the format width, so the column counter carries across newlines and resets only after a full set of columns has been read. During the index scan the parser records how many values appeared on the first data line inside a run; only when that equals the column count is the file known to be one record per line, and only then may byte ranges be split at newlines for parallel parsing.

A data line is one whose first non-blank character is a digit, a minus, a plus or a full stop. Everything else inside a run's byte range — free text, blank lines — is skipped silently, and a skipped line does not reset the wrapped-record column counter.

Lines are split on the line-feed character, then trimmed on the right of any run of carriage returns, spaces and tabs, and on the left of whitespace. Both Unix and Windows line endings are therefore handled; classic Mac files, which use a lone carriage return, are not.

7.5 Numeric encoding

Values are decimal text, never binary. The reader accepts rather more than `strtod` would, because Fortran output requires it.

The last row is a design decision worth stating plainly: an unreadable value becomes not-a-number, so a gap in the data leaves a gap in the curve, rather than a spurious zero that would silently distort both the plot and any statistic computed over it.

Mantissa digits accumulate into a 64-bit integer up to nineteen significant digits with the decimal exponent tracked separately, and are scaled once at the end against an exactly-representable table of powers of ten. Digits beyond the nineteenth increment the exponent in the integer part or are dropped in the fractional part. If an exponent marker turns out not to be followed by digits the

Form	Example	Notes
Plain	`1.5`, `-0.002`, `+3`	leading sign accepted
C exponent	`1.5e3`, `1.5E-3`	
Fortran double exponent	`1.234D+05`, `1.234d5`	`D` and `d` are read as `E`
Fortran bare-sign exponent	`1.234+05`	accepted only once mantissa digits have been seen
Non-numeric token	`NaN`, `Inf`, `*****`	becomes a quiet NaN — never zero

Table 7.1: Numeric forms the reader accepts.

pointer is rewound, so `1.5e` reads as `1.5` and leaves the `e` to the caller. The result is computed as a double and narrowed to a float for storage.

7.6 File extensions

The core is entirely extension-agnostic: it takes a path, maps it, and parses the content. The only place a suffix appears is the cache-path builder, which appends `.aoec` to the full source name. The extension list lives solely in the file dialog's filter, which accepts `*.aoe`, `*.dat`, `*.txt`, `*.res` and `*.out`. Those are the same format under different customary names; the two real dialects are the packed and indexed header forms, and they are distinguished by content rather than by filename.

CHAPTER 8

How the data is read

8.1 Memory mapping

The file is memory-mapped, read-only, in one view covering the whole file. There is no chunked or windowed mapping and no explicit page handling; demand paging is left entirely to the operating system, which is the right choice for a file that will be read column-by-column in an order the program cannot predict.

On POSIX the file is opened with `O_RDONLY | O_CLOEXEC`, sized with `fstat` — a size of zero or less is a failure — and mapped with `PROT_READ` and `MAP_PRIVATE`, with `madvise` asking for sequential access ahead of the index scan. On Windows the path is explicitly widened from UTF-8 to UTF-16 before `CreateFileW`, because the ANSI entry points failed silently on non-ASCII paths, and the share mode permits a solver to keep writing the file while it is open. There, asking for sequential access is a deliberate no-op: the flag would have to be set at open time and it measurably hurts the random column access that follows.

The modification time is read per platform — nanoseconds on POSIX, 100-nanosecond ticks on Windows — and returned as an opaque, monotonically increasing integer whose units differ between platforms and must never be interpreted. It is only ever compared for equality.

8.2 One index pass, then lazy parallel columns

Opening a file performs a single linear walk that records only the header names, the units, how many values appeared on the first data line of a run, and for each run its identifier, its unique key and the byte offsets at which its data begins and ends. *No sample data is read.* Opening a 578 MB file is therefore a scan, not a parse, and if a valid sidecar attaches the scan does not happen at all — headers, units, run identifiers, byte ranges and the values-per-line figure all come straight out of the sidecar's header.

Samples arrive later, per column and per run, the first time something asks for them. Each load maps the file afresh, clamps every recorded byte range to the file's *current* length — the solver this tool watches truncates and rewrites its output, so the index may have been built against a longer file — computes which columns are not already resident, divides the work into units, and dispatches them across the thread pool. Nothing already cached is re-read. Once parsed, the per-column chunks are stitched together outside the per-run locks, summarised in parallel, and published.

A byte range is split for parallelism only when the file is known to be one record per line, and only when the span exceeds 2 MiB; chunk boundaries are then advanced to the next line feed so that no line is ever split across two workers. The summary pass and the sidecar's copy path both run across the pool as well.

> It is worth saying explicitly, because it is often assumed of code this fast: there are no vector intrinsics and no OpenMP anywhere in the core. A search across the sources and build files for `immintrin`, `__m128`, `__m256`, `-mavx`, `-msse`, `-march=` and `-fopenmp` returns nothing. The parallelism is an ordinary thread pool; the only vector-like trick is scalar SWAR in the token skipper.

8.3 The inner loop

The innermost loop of the program is the one that turns bytes into floats, and it is written accordingly. A flat vector maps column index to output slot, because a hash lookup per token was measurably too slow. Output vectors are pre-reserved from the observed bytes-per-line of the first line rather than grown. The column counter is declared outside the line loop, which is what carries a wrapped record across newlines. Wanted columns go through the float parser; unwanted columns go through a much cheaper token skipper. Slack is returned to the allocator only when there is enough of it to be worth a reallocation.

Text-to-number conversion is hand-rolled rather than delegated to `strtod`, `strtof` or `std::from_chars`. The reasons are recorded in the header: `strtof` is correctly rounded but locale-aware and roughly an order of magnitude slower, and `std::from_chars` for floating-point types is missing from libstdc++ before version 11 and from older libc++. (`strtod` *is* used, but only for numeric literals in the formula parser, where performance is irrelevant.)

The token skipper uses the classic SWAR has-zero idiom over a 64-bit word loaded by `memcpy` rather than by a cast — a `reinterpret_cast` there is undefined behaviour that some compilers do miscompile at `-O3` — and locates the first separator byte with a count-trailing-zeros intrinsic. Big-endian builds fall back to a plain byte loop. The measured effect on a 500 MB file is about 2 GB/s against about 1.1 GB/s for a byte-at-a-time skip.

8.4 What happens to malformed input

The core throws no exceptions for malformed data and logs nothing. Every failure mode has a defined, quiet behaviour.

Situation	Behaviour
Non-numeric token	becomes a quiet NaN, token consumed; never zero
The float parser cannot advance	the rest of the line is abandoned
A line that does not start with a digit, sign or point	skipped; the column counter is *not* reset
A blank line inside a run	skipped
Wrong number of values on a wrapped record	the column counter drifts, and columns silently misalign from there on
The file shrank since it was indexed	byte ranges are clamped to the current length
The file cannot be mapped at load time	columns are marked loaded with an empty block, so the interface does not retry every frame and the channel reads as *absent* rather than as a line of zeros
Empty or zero-length file	mapping fails, so opening fails
No `*PARAMETRES` block	opening fails
More units than headers	the unit list is resized to the header count
The sidecar disagrees with the dataset's shape	the sidecar is ignored and the text parse takes over

Table 8.1: Complete list of malformed-input behaviour.

8.5 The `.aoec` sidecar

The first time a file of 8 MB or more is opened, the same background task that opened it also writes a sidecar beside it. Thereafter opening that file skips the index scan and loading a column becomes a copy out of a mapping.

8.5.1 Layout

The file begins with a fixed 128-byte header, followed by one 16-byte record per column, one 24-byte record per run, then the sample data aligned to a 64-byte cache line. The header carries the magic `"AOECACHE"` — eight bytes, not NUL-terminated — a format version of 1, a byte-order mark of `0x01020304`, the column and run counts, the values-per-line figure, and the source fingerprint. What is stored is column-major `float32` sample arrays, plus the complete index — headers, units, run identifiers, run byte ranges in the source, values per line — plus per-parameter-per-run summary statistics: minimum, maximum, whether any value is finite, whether there are gaps, and whether the channel is monotonic. No text is stored, no decimated form, and no formula results.

8.5.2 Deciding whether it is still valid

Three independent properties are compared for exact equality, and all three must match.

Field	Catches	Would miss on its own
size	truncation, extension, append	an in-place rewrite of the same length
modification time	an in-place rewrite	anything that restores timestamps: `rsync -times`, a restore from backup, some build systems
content hash	a rewrite that preserved both of the above	nothing — it is the backstop

Table 8.2: The three-part source fingerprint.

The hash is 64-bit FNV-1a, chosen explicitly for being verifiable at a glance: it guards against a file changing underneath the program, not against an adversary crafting a collision. It is seeded with the file's length, so a file that is a strict prefix of another cannot collide with it, and the length is taken from the same open handle that the bytes are read through, so length and content always describe the same file. Files of 4 MiB or less are hashed in full. Larger files are sampled: sixty-four evenly spaced windows of 64 KiB, the first at offset zero and the last ending at the end of the file, with each window's offset folded into the hash before its bytes so that identical blocks moved to a different place in a rearranged file do not cancel out. The cost of validating a sidecar is therefore capped at about 4 MiB of reading however large the source is — which is exactly why the advantage of a sidecar widens as files grow.

8.5.3 Where sidecars live

Two locations are tried, nearest first. Beside the source, the sidecar is the full source name with `.aoec` appended — `run.aoe` becomes `run.aoe.aoec`, keeping the original extension so it can never collide with a differently-suffixed file of the same stem. Failing that, a per-user cache directory is used, with the full source path hashed into the filename so that two files called `results.aoe` in different directories do not share a cache.

Platform	Per-user cache directory
Windows	`%LOCALAPPDATA%\\aoeViewer\\cache`, falling back to `%APPDATA%`, then `%TEMP%`
macOS	`$HOME/Library/Caches/aoeviewer`
Linux and others	`$XDG_CACHE_HOME/aoeviewer`, else `$HOME/.cache/aoeviewer`

Table 8.3: The fallback location, which `AOE_CACHE_DIR` overrides entirely.

Each candidate is tried in turn and the first that both opens and matches is used; one that fails to match is skipped rather than treated as fatal, because the other location may hold a current one.

Caching can be switched off entirely by setting `AOE_CACHE` to `0`, `false`, `no` or `off`; the variable is read once per process, because an environment lookup on Windows is a system call.

8.5.4 What it buys

On a 73 MB generated file of 1200 runs by 1000 rows by 6 channels, measured end to end:

Operation	From text	From sidecar
Open (index)	213 ms	22 ms
Preload 2 channels across 1200 runs	81 ms	7 ms
Fetch 1200 curves	6 ms	5 ms
ISO scan	7 ms	0.3 ms
Total	**306 ms**	**32 ms**

Table 8.4: Roughly a tenfold improvement. Writing the sidecar took 0.24 s once, and it occupies 27.7 MB against a 73.1 MB source — `float32` is more compact than the decimal text it came from.

The two structural wins are that opening does not scan, and that loading a column becomes a copy out of a mapping plus a copy of the precomputed summary, so the summarising pass does not run at all.

`MainWindow.CACHE_THRESHOLD_BYTES` is `8 * 1024 * 1024`, and the decision to write a sidecar is made there. Below that size, parsing is quick enough that a sidecar would save nothing measurable and would only leave an extra file beside the results. The consequence is that a script using the Python API gets no sidecar unless it calls `build_cache()` itself. Every reason to skip is silent: a read-only share, a full disk, caching switched off.

8.6 The memory budget

The core owns all sample memory and reports how much it is holding. When that figure exceeds the configured limit — 2048 MB by default, floored at 64 MB — the interface asks the core to trim to 80% of the cap, which gives the eviction some hysteresis instead of trimming on every redraw. Eviction is least-recently-used by column, and is always safe: an evicted column is simply re-read the next time something asks for it.

Eviction can never free a buffer a visible curve is reading from. Arrays handed to Python are read-only views of the core's memory rather than copies, and each carries a pin object as its NumPy base which keeps the underlying block alive for as long as any view exists. That is also why an eviction invalidates the fast redraw path: what was resident before says nothing about what is resident now, so the next redraw re-checks.

CHAPTER 9

Computed parameters: the formula language

A computed parameter is a named expression over the channels that exist, evaluated in the core, per run, and thereafter indistinguishable from a channel the file carried. This chapter is the complete language reference.

9.1 How an expression is executed

An expression is compiled once into a flat bytecode and evaluated by a stack machine vectorised over rows. It is deliberately not an abstract syntax tree walked per sample. Compilation happens once per formula rather than once per run; the bytecode stores variable *slots* and data is bound at evaluation time, so no pointer can dangle when a later column load reallocates a vector. There is no per-node allocation and no virtual call.

The evaluator tracks whether each stack entry is a broadcast scalar or a real buffer, so `2 * 9.81` collapses to a single float rather than filling a million-element array, and buffers are drawn from a reused pool so that a deep expression allocates a handful of buffers rather than one per operator.

9.2 Operators

Level	Operators	Associativity	Meaning
1 (lowest)	`||`	left	logical or
2	`&&`	left	logical and
3	`== !=`	left	equality
4	`<= >= < >`	left	comparison
5	`+ −`	left	addition
6	`* / %`	left	multiplication, division, remainder
7	unary `− + !`	right	sign and logical not
8 (highest)	`^`	**right**	power

Table 9.1: Precedence, lowest to highest. Because `^` is right-associative, `2^3^2` is 512 rather than 64.

Three behaviours are worth knowing before they surprise you. Division by zero yields not-a-number, never an infinity: the reason is stated twice in the source, that an infinity silently destroys the plot's y range whereas a NaN simply breaks the line where the data is bad. The same applies to `%` and to `mod`. Comparisons yield exactly `1.0` or `0.0`, so they can be multiplied into arithmetic. And the logical operators do not short-circuit — both sides are always evaluated — which matters only in that a costly right-hand side is not skipped.

The power operator binds tighter than unary minus on its left, so `-2^2` is $-(2^2)$, matching engineering calculators, while `2^-3` still works because the exponent side is parsed as a unary expression. Logical not never swallows an inequality, and `!NaN` is `0.0`.

9.3 Element-wise functions

These operate one sample at a time.

Function	Behaviour
`abs(v)`	absolute value
`sqrt(v)`	NaN for a negative argument
`sin cos tan`	as the C library
`asin(v) acos(v)`	NaN outside $[-1, 1]$
`atan(v)`	arctangent
`exp(v)`	exponential
`log(v) ln(v)`	**natural** logarithm; both names map to the same operation, and a non-positive argument gives NaN
`log10(v)`	base-10 logarithm; non-positive gives NaN
`floor ceil round`	as the C library
`sign(v)`	-1, 0 or $+1$; NaN gives 0
`deg(v)`	radians to degrees
`rad(v)`	degrees to radians
`min(a,b) max(a,b)`	per-sample smaller or larger; NaN if either is NaN
`atan2(a,b)`	two-argument arctangent
`pow(a,b)`	as `a^b`
`mod(a,b)`	remainder; NaN when `b` is zero
`hypot(a,b)`	$\sqrt{a^2 + b^2}$ without intermediate overflow
`clamp(x, lo, hi)`	constrains `x`; NaN stays NaN
`if(cond, a, b)`	`a` when `cond` is non-zero and not NaN, else `b`. A NaN condition takes the false branch, and both branches are always evaluated

Table 9.2: Every element-wise function, spelled as the parser expects it.

9.3.1 The three logarithms

Which logarithm a name gives is the easiest thing in this language to assume wrongly, since a pocket calculator's `log` is base ten and this one is not. There are two functions under three names.

Name	What it computes
`log(x)`	The **natural** logarithm, base $e = 2.71828\ldots$ — *not* base ten. This follows C and Fortran rather than a calculator keypad.
`ln(x)`	Exactly the same function as `log`, under the name most engineering texts use. Write whichever reads better; the two give identical results.
`log10(x)`	The base-ten logarithm. This is the one for decibels, orders of magnitude, and anything you would read off a logarithmic axis.

Table 9.3: The logarithms, spelled out because the defaults are not obvious.

All three return NaN for an argument that is zero or negative rather than an infinity, for the reason given above: a NaN breaks the line where the data is out of domain, an infinity would destroy the plot's y range. The inverse of `log` and `ln` is `exp(x)`; the inverse of `log10` is `pow(10, x)`.

```
DECIBELS = 20 * log10(AMPLITUDE / AMP_REF)
DECADES  = log10(ALTITUDE)
TAU      = -1 / deriv(ln(ENERGY), TIME)
```

The **Log Y** checkbox in **STYLE** draws an existing channel on a base-ten axis and leaves the data untouched. Use that to *look* at a channel logarithmically. Use these functions when you want the logarithm itself as a channel — to take a slope off it, say. All of this is now stated in the **Help** dialog, which previously listed `log` and `log10` and mentioned neither `ln` nor which base `log` used.

9.4 Whole-series functions

These cannot be evaluated one sample at a time — a derivative needs its neighbours, a filter needs its own previous output — so they operate on the finished buffer.

Function	Behaviour
`deriv(Y, X)`	dY/dX by central differences, one-sided at the two ends, accumulated in double precision. A zero step gives NaN rather than an infinity. A single sample gives zero
`integ(Y, X)`	cumulative trapezoidal integral starting at zero. NaN steps contribute nothing but the running total is still written out
`lowpass(Y, X, fc)`	first-order recursive low pass with the cutoff in hertz. The timestep is taken from `X` at every sample, so uneven sampling is handled correctly. A non-positive cutoff performs no filtering at all rather than dividing by zero
`highpass(Y, X, fc)`	the matching first-order high pass; the first output sample is zero
`movavg(Y, N)`	centred moving average over `N` samples, computed by prefix sum so the cost is independent of the window size. NaN samples are excluded from both the sum and the count; an all-NaN window gives NaN
`delta(Y)`	`Y` minus its first sample, so a leading NaN makes the whole channel NaN
`min(Y) max(Y) mean(Y)`	reductions to one number, accumulated in double and ignoring NaN and infinity, then held flat across every sample so the result combines with per-sample channels by ordinary arithmetic

Table 9.4: The series functions and the three reductions.

Overload resolution is by argument count, not by name. `max(NZ)` is the run's peak; `max(NZ, NY)` is the per-sample larger of two channels. Each run is reduced separately, which is what makes `NZ / max(NZ)` normalise every run against its own peak. If a name exists but no definition has that arity, the error names the counts that would have worked — *min() takes 1 or 2 arguments, not 3*.

Passing a scalar where a series function expects its `X` argument fills the result with NaN, because a constant x would make every step zero.

9.5 Constants and the lookup form

Three constants are recognised, case-insensitively: `pi`, `e` and `nan`. There is no `inf`, and no `true` or `false` — comparisons already yield one and zero.

The lookup form reads one channel at the moment another channel satisfies a condition, and broadcasts that single value across the whole series:

```
NAME[COND=value]      # NAME at the sample where COND is closest to value
NAME[COND=first]      # NAME at the first sample where both are real
NAME[COND=last]       # NAME at the last sample where both are real
```

For a numeric target the sample minimising the absolute difference wins; samples whose condition is NaN are skipped, though a NaN *value* at the winning condition is returned as it stands. For `first` and `last` the scan runs from the respective end looking for the first index at which both the condition and the value are real numbers, so leading and trailing gaps are skipped and the lookup returns an actual sample rather than NaN. If nothing is found the result is NaN.

9.6 Worked examples

```
LOAD_FACTOR  = NZ * 9.80665
PITCH_RATE   = deriv(THETA, TIME)
DISTANCE     = integ(VTAS, TIME)
CLIMB_RATE   = deriv(ALTITUDE, TIME)
ALPHA_SMOOTH = lowpass(ALPHA, TIME, 2.0)
ALPHA_NOISE  = highpass(ALPHA, TIME, 5.0)
DRIFT        = HEADING - HEADING[TIME=first]
ALT_GAINED   = ALTITUDE[TIME=last] - ALTITUDE[TIME=first]
HIGH_MACH    = if(MACH > 0.8, ALPHA, nan)
ENERGY       = 0.5 * MASS * (VX^2 + VY^2)
NZ_NORM      = NZ / max(NZ)
NZ_ENVELOPE  = max(NZ, NY)
ALPHA_EXCESS = ALPHA - mean(ALPHA)
```

`HIGH_MACH` is worth a second look: substituting NaN for the samples you do not want breaks the line there, which is how you isolate a flight phase without editing the data.

9.7 Names, dependencies and errors

Any identifier that is not a call, a lookup or one of the three constants becomes a variable. Names are resolved after alias resolution, and a name must be either a physical column of some open file or another computed channel — otherwise the definition is refused with *unknown parameter 'X'*.

Formulas may reference other computed parameters, and the dependency graph is checked at definition time rather than discovered at plot time. The new definition is installed tentatively, the graph is walked with a depth limit of 32, and on detection the previous definition is restored and the error reads *'NAME' would depend on itself*. Exceeding the depth limit is reported the same way. Removing a parameter removes everything defined in terms of it, and the interface reports which.

Every variable is bound once, up front; a variable that resolves to nothing is a hard failure reading *no data for parameter 'X'*. The sample count is the shortest of the bound channels, and is never allowed to exceed it — the engine never reads past the shortest channel involved. A formula with no variables at all, such as `pi` or `2 + 3 * 4`, would otherwise have no length to fill, so the run's first column is consulted for a row count, falling back to a single row.

Validation, which is what runs as you type in the **COMPUTE** tab, compiles the expression and checks every variable against the set of known names without touching any data. The distinct compile-time messages are *empty expression*, *unexpected trailing text*, *expected ')'*, *malformed number*, *unexpected character*, *expected ',' or ')' in function arguments*, *unknown function 'X'*, the arity message, four specific lookup messages, and *invalid expression* as a last resort. The lookup messages are unusually helpful and worth quoting:

```
expected a parameter name inside [ ]
expected '=' in lookup, e.g. SPEED[TIME=0.5]
expected a number, 'first' or 'last' in lookup,
    e.g. SPEED[TIME=0.5] or SPEED[TIME=last]
expected ']' in lookup
```

Results are cached per run and per name, and *failures are cached too*, as an empty block. Without that the interface would retry the same doomed evaluation on every repaint; and the tempting alternative — substituting a zero — would turn a channel that could not be computed into a flat line that looks like data.
A computed channel also carries no unit. Deriving one through a formula would mean dimensional analysis, and a wrong unit on an axis is worse than none.

CHAPTER 10

The algorithms behind the plots

10.1 Decimation: drawing what the screen can show

A tile with twenty-five runs of half a million samples cannot hand two and a half million points to a plotting library and stay interactive. Nor can it keep every thousandth sample: stride sampling silently deletes every spike between the samples it keeps, and on flight-test data that is exactly the part an engineer is looking for. aoeViewer therefore uses **min/max bucketing**.

The visible index span is divided into exactly `max_points` buckets by index, so bucket sizes differ by at most one sample. Within each bucket the minimum and maximum y are found, skipping NaN, and both are emitted in x order — swapped if necessary so the polyline never doubles back. Where a bucket's extremes coincide, one point is emitted; where a bucket is entirely NaN, a single NaN is emitted at the bucket's first x, so the line breaks visibly instead of interpolating across missing data. The output is therefore at most $2 \times$ `max_points` points. Below a count of twice the budget there is nothing to gain, and the span is copied through verbatim.

> Every bucket's extremes survive, so the drawn envelope is pixel-accurate: no peak or trough can be lost at any zoom level, because it is either the minimum or the maximum of whatever bucket it falls into. What is not preserved is the shape *between* extremes within a bucket, nor the exact ordering of intermediate samples. NaN gaps are preserved as gaps, at bucket granularity.

10.1.1 Selecting the visible window

Before bucketing, the visible x window must be turned into an index span, and this is where a subtlety lives. When x is known to be non-decreasing, two binary searches find the span in logarithmic time. When it is not — and a phase-plane plot such as Mach against altitude over a climb and descent is not — a *full scan* finds the outermost indices whose x falls inside the window and keeps everything between them.

The reason is a real defect the scan fixes. On a climb-and-descend trajectory x rises and then falls, so a binary search brackets only the first crossing and the entire descending half of the trajectory silently disappears — differently depending on where you have panned to. One sample is also kept on each side of the window, so the curve is drawn to the plot border rather than stopping just short of it.

10.1.2 The pixel budget

The front end asks for roughly two points per horizontal pixel — one for each bucket's high and low — so a 600-pixel tile draws about 1200 points whether the channel holds ten thousand samples or ten million. A second ceiling of 1 500 000 points applies across all the curves of one tile together, so that a page with many curves cannot overwhelm Qt.

That second ceiling is a named function rather than an inline expression for a reason recorded in the source: written as a bare division it acted as a *floor* at moderate curve counts, *raising* the target from 1800 to 10 000 at seventy-five curves and pushing one and a half million points into Qt on every pan.

The visible window is padded by five per cent on each side so the curve stays drawn to the tile's edges while panning. The quoted end-to-end effect on a heavy page is a sixty-fold reduction: 15.9 million samples become 266 thousand drawn points across 225 curves.

10.1.3 Statistics over the same window

The statistics shown in **INSPECT** use Welford's one-pass method rather than accumulating a sum of squares, because a sum of squares loses precision on a channel with a large offset — altitude in feet, squared, overflows single precision quickly. The reported standard deviation is the sample standard deviation and is zero for a single sample. NaN and infinite samples are skipped. On a non-monotonic x with a window given, every sample is tested individually rather than bracketing a span, for exactly the phase-plane reason above.

10.2 The redraw cache

The decimation result for a group of curves is tagged with a *resample key*: the quantised lower edge of the view, the span rounded to twelve significant figures, the point target, and whether the axis is logarithmic. A group whose key is unchanged is skipped entirely.

Both halves of that key are necessary. Quantising only the endpoints hides every zoom whose lower edge scales with its width — the windows $0 \ldots 1$ and $0 \ldots 0.25$ quantise into the same bucket — so zooming in around the origin gained no resolution. Rounding to twelve significant figures is needed because a view range read back from a plot wobbles in the last bits of a double, which would otherwise invalidate the cache on every mouse move.

A second cache decides whether the drawn items themselves can be kept. Each group records a signature over its curves: the curve's identity, and the *address and length* of the x and y buffers behind it. A reloaded file, or a column that was evicted by the memory cap and re-read, lands at a different address and so compares unequal. When the signature matches, the existing graphics item is adopted and merely restyled; the group keeps its resample key, so no points are recomputed either. The net effect is that restyling a curve, toggling the grid, dragging a marker or returning to a page redraws *no points at all*.

> Curves that share a pen cannot be told apart on screen, so they are concatenated into a single polyline with one NaN between consecutive runs and drawn with a connect rule that breaks the line there. Twelve hundred runs times two parameters — 2400 graphics items — becomes about twelve. Everything except painting is still per curve: the crosshair readout, click-to-inspect, marker dots, area segments and the CSV export all need to know which run and which parameter a sample belongs to.

Two of pyqtgraph's own optimisations are deliberately switched off. Clip-to-view assumes uniformly spaced, sorted x and slices by index arithmetic, which is false for a concatenated multi-run polyline and silently cut most runs out of the item. Automatic downsampling is stride-based and would drop the transients that min/max bucketing exists to preserve. Antialiasing is off on two-dimensional curves for the same reason it is not needed: with min/max decimation they are already at pixel resolution.

10.3 ISO grouping

Automatic grouping divides the observed range of the distinct values into evenly spaced intervals. The extremes are found ignoring NaN and infinity; the step is computed in double precision; and the last interval's upper bound is set to the observed maximum exactly, rather than recomputed,

so accumulated floating-point error cannot leave the largest sample outside every group. A sweep with a single distinct value yields one group whose bounds are equal, rather than several degenerate empty ones.

10.3.1 Boundaries

Every interval except the last is half-open, $[\min, \max)$; the last is fully closed, $[\min, \max]$. A value that is not a number belongs to no group. Two clamping fallbacks catch a value sitting a hair outside the extreme bounds through rounding — without them the maximum sample of a sweep can vanish — while a value between two non-adjacent manual intervals still belongs to none.

> The core header describes group bounds as inclusive at both ends, and the Python-side helpers implement inclusive-at-both-ends with the first claiming group winning. The core's own `iso_group_index` is half-open except for the last interval. The two agree on every value except one sitting exactly on a shared edge, which the core assigns to the later interval and the interface to the earlier one. In practice this is invisible — the edge is a value no sample holds, because the edges are derived from the observed range rather than from the samples — but it is a genuine discrepancy.

Where grouping is configured but the interval list is empty, nothing is filtered. Nothing sorts the interval list after the user has edited it, so overlapping manual bounds are resolved by the first match in the order given.

When a new file is opened, an automatically grouped parameter is rebuilt over the new range and the active flags are carried across by position, so deselections survive. A manually grouped parameter keeps its bounds, and only names that still look automatically generated — empty, or containing the exact separator `" – "` — are refreshed, so hand-typed labels are preserved.

10.3.2 How values are written

ISO values are formatted to two decimals with trailing zeros trimmed, so `0.750000` becomes `0.75` and `3.000000` becomes `3`. Values that round to zero, and values of a million or more, are written as a power of ten with a compacted exponent: `1.20e+06` becomes `1.2e6`. A negative zero is normalised to `0`.

The small-end boundary is decided by rounding rather than by a threshold, and the reasoning is worth knowing because it affects the labels you see. A value arrives as a single-precision float, so the literal `0.01` is stored as `0.00999999978`; comparing *that* against 10^{-2} would send `0.01` itself to scientific notation — the one value a Mach or tolerance sweep is most likely to contain. Asking instead whether the rounded form is still non-zero puts the boundary where it belongs. The large end genuinely is a threshold, and it is about label length: past a million the integer part alone is longer than the value written as a power of ten.

10.4 Legend order, and why it stopped shuffling

A legend row is ranked by an intrinsic property of what it describes, not by the order the curves happened to be walked in. In ISO mode a grouped label ranks by its position in the grouping and an ungrouped label by the value itself; the lowest rank wins when two runs share a label. Groups are then sorted by a tuple — ISO rank, parameter order, file, run in ISO mode; parameter order, file, run when colouring by parameter; file, run, parameter order when colouring by simulation.

Before this, the order was the order in which each group was first seen while walking the selection, so unticking a single run could reshuffle *0–10, 10–15, 15–20* into *15–20, 0–10, 10–15*. Nothing about the data had changed; only the walk had.

10.5 Logarithmic axes

pyqtgraph's logarithmic mode does not scale the axis: it plots the base-10 logarithm of the samples on a linear axis and relabels the ticks. Everything crossing the view boundary is therefore in log units, while a tile's stored minimum and maximum are, and must remain, in data units. A dedicated module owns that conversion, and it exists because of a specific defect: ticking **Log Y** on a channel spanning 1 to 1000 framed the view at log-space 1 to 1000 — data 10^1 to 10^{1000} — while the curve sat at log values 0 to 3. The tile looked empty, double-clicking recomputed the same answer, and those mixed numbers then propagated as a variable-wide limit onto tiles that were not in logarithmic mode at all.

Converting a value to view space returns nothing at all when the value is non-positive, so the caller must decide what to do rather than being handed a silently wrong number. When the low end of a requested range has no logarithm, the frame is taken as three decades below the high end rather than refusing to frame at all. At draw time non-positive samples are blanked to NaN so the drawable part of the curve still shows.

10.6 The three-dimensional projection

The projection is orthographic, and deliberately so: these plots exist to have values read off them, and under perspective the same interval covers different distances depending on where it sits in the frame.

Each of the three channels is normalised independently onto $[-0.5, 0.5]$, so channels with unrelated units stay readable together; a degenerate or non-finite range collapses to zero, which puts a channel that never moves in the middle of the cube. The path is then rotated about the vertical axis by the azimuth, tilted by the elevation, and depth is dropped:

$$r_x = x\cos A - y\sin A, \qquad r_y = x\sin A + y\cos A,$$
$$\text{screen}_x = r_x, \qquad \text{screen}_y = z\cos E - r_y\sin E,$$

with depth $z\sin E + r_y\cos E$ retained only for ordering what is drawn in front of what.

A projected path cannot be min/max decimated, because that needs a monotonic x and a trajectory through space is monotonic in none of the three axes. Plain striding is the honest option here, and it is what is used, against a budget derived from the tile width as in the two-dimensional case.

Rotating does not rebuild the tile. Only the grid, the triad and the projection are redone; the channel reads, the bounds and the thinning are left alone, which is what makes dragging to rotate smooth rather than a sequence of full reloads.

The back-wall grid picks, for each axis, the face lying *behind* the data from the current viewpoint, so the grid never draws over the trajectory and swaps faces as you turn it. Its tick values are chosen in steps of 1, 2, 2.5 or 5 times a power of ten, counted rather than accumulated so they cannot drift outside the box, and the whole grid is drawn as a single graphics item. The orientation triad is parented to the view box rather than to the data, so panning and zooming never reach it and it keeps a constant size; its axes follow the universal convention of red, green and blue for x, y and z, and are painted back to front by depth.

CHAPTER 11

The C ABI

Everything above the core is written without a C++ compiler, and the reason is this interface: sixty exported functions taking only C types and opaque handles. It is declared in `core/include/aoe/aoe_c_api.h`.

11.1 Handles, versioning and errors

```
typedef struct aoe_session aoe_session;   /* opaque; the API's whole context */
typedef struct aoe_pin     aoe_pin;       /* opaque; keeps a sample block alive */
```

A pin holds *the sample block itself*, not a path back to the session, so releasing one after the session has been destroyed is well defined. That matters because on the Python side pins are released by the garbage collector, in its own order and at its own time.

The only version marker is `aoe_version()`, which returns a string — currently `"2.0.0"`. There is no numeric ABI-version symbol and no version macro in the header.

Errors are reported per session. `aoe_last_error()` returns the message from the most recent failure, or the empty string. It is cleared at the start of every mutating call but not by read-only calls, so it means *the last failure since the last mutating call* rather than strictly the last call. Failure is signalled by a sentinel that depends on the return type: -1 for a function returning an index, 0 for a boolean or a count, an empty string — never a null pointer — for a string, a null pointer for a float array, and zero for a size. Index arguments out of range are errors rather than crashes, and a null session pointer is handled without being dereferenced.

> Every session-taking entry point funnels through a guard that catches `std::exception`, putting its message into the session's error slot, and any other exception as *unknown error*. Unwinding out of a C function is undefined behaviour, and with `ctypes` on the other side it would take the interpreter down.

11.2 Thread safety

The session carries a reader/writer lock — an SRWLOCK on Windows, a `pthread_rwlock_t` elsewhere. Read-only entry points take it shared, so many concurrent readers are allowed; mutating ones take it exclusively and clear the error string. The mutating set is: adding, removing, clearing and reloading files; setting a file or run display name; setting and removing aliases; setting and removing computed parameters; building and discarding a cache; and the two calls that write into the session's scratch string, `aoe_resolve` and `aoe_iso_format`, which are logically reads but take the write lock because of that scratch.

The pure-mathematics functions — decimation, the series helpers, nearest-index, and the ISO grouping arithmetic — take no session at all and are fully re-entrant on caller-owned buffers. Below the ABI, each run has its own mutex guarding its column vector and its map of computed results, which is what lets the thread pool fill different runs concurrently; an attached cache file is immutable once open and needs no locking.

`aoe_cache_path` takes only the shared lock yet writes the session's scratch string. Two concurrent readers can therefore each invalidate the other's returned pointer. Copy returned strings immediately — which `ctypes` does automatically when the result type is declared as a C string, so the Python layer is not exposed to this.

11.3 Function reference

In the table below, *lock* is the lock the call takes; *session* in the note means memory owned by the session and valid until the next call that changes the thing it came from; *core buffer* means a zero-copy alias of a column buffer.

Function	Parameters	Returns	Lock
`aoe_version`	*none*	`const char*`	—
static literal, always valid			
`aoe_worker_count`	*none*	`int`	—
value			
`aoe_session_create`	*none*	`aoe_session*`	—
caller owns; free with `aoe_session_destroy`. *Failure:* `NULL`			
`aoe_session_destroy`	`aoe_session*`	`void`	—
consumes the handle; `NULL` safe (`delete nullptr`)			
`aoe_last_error`	`s`	`const char*`	—
session; valid until the next call. *Failure:* `""`			
`aoe_add_file`	`s, const char* path`	`int` index	write
—. *Failure:* `-1`, error set			
`aoe_remove_file`	`s, int file`	`void`	write
invalidates every pointer from that file			
`aoe_clear_files`	`s`	`void`	write
invalidates all file-derived pointers; **aliases and computed definitions survive**			
`aoe_file_count`	`s`	`int`	read
value. *Failure:* `0`			
`aoe_file_name`	`s, int file`	`const char*`	read
session. *Failure:* `""`			
`aoe_file_path`	`s, int file`	`const char*`	read
session. *Failure:* `""`			
`aoe_stale_files`	`s, int* out, int max`	`int` count	read
writes $\leq$ `max` into `out`; `out=NULL/max=0` just counts; **returns the full count**. *Failure:* `0`			
`aoe_reload_file`	`s, int file`	`int` 1/0	write
drops cached samples and computed results for that file. *Failure:* `0`			
`aoe_set_file_name`	`s, int file, const char* name`	`void`	write
display only; empty/NULL name ignored			
`aoe_set_sim_name`	`s, int file, int sim, const char* name`	`void`	write
display only; **`unique_key` untouched**, so colours survive			
`aoe_sim_count`	`s, int file`	`int`	read

continues

continued from the previous page

Function	Parameters	Returns	Lock
value. *Failure:* `0`			
`aoe_sim_id`	`s, int file, int sim`	`const char*`	read
session. *Failure:* `""`			
`aoe_sim_name`	`s, int file, int sim`	`const char*`	read
session; falls back to `id` when the display name is empty. *Failure:* `""`			
`aoe_sim_key`	`s, int file, int sim`	`const char*`	read
session; the **identity key** styles/workspaces are built on. *Failure:* `""`			
`aoe_header_count`	`s, int file`	`int`	read
value. *Failure:* `0`			
`aoe_header`	`s, int file, int index`	`const char*`	read
session. *Failure:* `""`			
`aoe_header_unit`	`s, int file, int index`	`const char*`	read
session; parallel to `aoe_header`; **never NULL**. *Failure:* `""`			
`aoe_param_unit`	`s, const char* name`	`const char*`	read
session; searches every open file, follows aliases; `""` for computed. *Failure:* `""`			
`aoe_param_count`	`s`	`int`	read
union of headers + computed + aliases, **sorted**. *Failure:* `0`			
`aoe_param`	`s, int index`	`const char*`	read
session. *Failure:* `""`			
`aoe_set_alias`	`s, const char* alias, const char* target`	`void`	write
no-op if either empty or `alias == target`			
`aoe_remove_alias`	`s, const char* alias`	`void`	write
—			
`aoe_resolve`	`s, const char* name`	`const char*`	**write**
session `scratch` — **invalidated by the next scratch-writing call**. *Failure:* `""`			
`aoe_set_computed`	`s, const char* name, const char* expression`	`int` 1/0	write
clears cached computed results in every file. *Failure:* `0`, error set			
`aoe_remove_computed`	`s, const char* name`	`int` count	write
removes transitively; names readable via #31. *Failure:* `0`			
`aoe_removed_name`	`s, int index`	`const char*`	read
session `removed` vector; valid until the next `aoe_remove_computed`. *Failure:* `""`			
`aoe_is_computed`	`s, const char* name`	`int` 1/0	read
resolves aliases first. *Failure:* `0`			
`aoe_computed_source`	`s, const char* name`	`const char*`	read
session; the stored expression text. *Failure:* `""`			
`aoe_validate_formulas`	`, const char* expression`	`int` 1/0	read
parses + checks variables, **evaluates nothing**. *Failure:* `0`, error set			

continues

continued from the previous page

Function	Parameters	Returns	Lock
`aoe_channel`	`s, int file, int sim, const char* name, size_t* out_count`	`const float*`	read

core buffer, zero-copy. Valid only until the next eviction — anything that keeps it must use #36. *Failure:* `NULL`, `*out_count = 0`

`aoe_channel_pinned`	`s, int file, int sim, const char* name, size_t* out_count, aoe_pin** out_pin`	`const float*`	read

caller owns `*out_pin`; the samples stay alive regardless of the memory cap until `aoe_pin_release`. `*out_pin = NULL` when there is nothing to return. *Failure:* `NULL`

`aoe_pin_release`	`aoe_pin*`	`void`	—

frees the pin; `NULL` safe

`aoe_pin_sorted`	`aoe_pin*`	`int` 1/0	—

precomputed at parse time — **free**; 1 = samples non-decreasing (precondition of any binary search over x). *Failure:* `0`

`aoe_pin_bounds`	`aoe_pin*, float* out_min, float* out_max`	`int` 1/0	—

precomputed extent; this is what autoscaling needs without rescanning a million samples per curve per replot. *Failure:* `0`

`aoe_pin_count`	`aoe_pin*`	`size_t`	—

value. *Failure:* `0`

`aoe_preload`	`s, int file, const int* sims, int n_sims, const char* const* names, int n_names`	`void`	read

loads every physical column the names need, in one parallel pass; `sims=NULL/n_sims=0` means **all**

`aoe_decimate`	`x, y, count, x_min, x_max, max_points, out_x, out_y`	`size_t` written	—

caller owns all buffers; out needs $\geq$ `2*max_points`. Uses the whole series when `!(x_min < x_max)`; implies `x_sorted = -1`. *Failure:* `0`

`aoe_decimate_ex`	as #42 plus `int x_sorted` before the outputs	`size_t`	—

`x_sorted`: 1 sorted, 0 not, −1 detect. **Pass 0 for phase planes**. *Failure:* `0`

`aoe_series_sorted`	`const float* values, size_t count`	`int` 1/0	—

caller buffer. *Failure:* `0`

`aoe_series_bounds`	`values, count, float* out_min, float* out_max`	`int` 1/0	—

caller buffers. *Failure:* `0`

`aoe_nearest_index`	`x, count, target`	`size_t`	—

returns `0` for an empty series — **indistinguishable from a genuine hit at index 0**. *Failure:* `0`

`aoe_series_stats`	`x, y, count, x_min, x_max, double* out`	`int` 1/0	—

continues

continued from the previous page

Function	Parameters	Returns	Lock
writes 7 doubles: min, max, mean, stddev, first, last, count. `x=NULL` or `x_min >= x_max` → whole series. *Failure:* `0`			
`aoe_series_stats_ex`	as #47 plus `int x_sorted` before `out`	`int` 1/0	—
same `x_sorted` contract as #43. *Failure:* `0`			
`aoe_resident_bytes`	`s`	`size_t`	read
bytes of sample data cached across every open file. *Failure:* `0`			
`aoe_trim_memory`	`s, size_t keep_bytes`	`size_t`	read
frees LRU columns until ≤ `keep_bytes`; returns the total afterwards. **Always safe** — evicted data is re-read on demand. *Failure:* `0`			
`aoe_cache_enabled`	*none*	`int` 1/0	—
`0` when `AOE_CACHE` switches caching off			
`aoe_cache_active`	`s, int file`	`int` 1/0	read
1 when a valid sidecar is attached and serving reads. *Failure:* `0`			
`aoe_cache_path`	`s, int file`	`const char*`	read
session `scratch` — see the concurrency caveat in §7.3. *Failure:* `""`			
`aoe_build_cache`	`s, int file`	`int` 1/0	**write**
costs a full parse — **call off the UI thread**. Memory bounded to one parameter across every run. *Failure:* `0`, error set			
`aoe_discard_cache`	`s, int file`	`int` 1/0	write
detaches **and deletes** every candidate sidecar; 1 when none remains. *Failure:* `0`			
`aoe_iso_values`	`s, int file, const char* name, float* out, int max`	`int` count	read
writes ≤ `max`; **returns the full count**; NaN where absent. *Failure:* `0`			
`aoe_iso_distinct`	`s, const char* name, float* out, int max`	`int` count	read
ascending, NaN-free, de-duplicated across every open file; `out=NULL` to learn the count first. *Failure:* `0`			
`aoe_iso_auto_group`	`const float* values, int count, int group_count, float* out_bounds, int max_groups`	`int` groups	—
`out_bounds` receives **2 floats per group** (min, max). *Failure:* `0`			
`aoe_iso_group_index`	`float value, const float* bounds, int group_count`	`int`	—
`bounds` is 2 floats per group. *Failure:* `-1`			
`aoe_iso_format`	`s, float value`	`const char*`	**write**
session `scratch`; valid until the next call on this session. *Failure:* `""`			

Three of these functions — `aoe_stale_files`, `aoe_iso_values` and `aoe_iso_distinct` — return the *full* count rather than the number actually written, so a caller can size a buffer with one call and fill it with a second.

`aoe_channel` hands back a zero-copy pointer into a column buffer which is valid only until the next eviction; anything that intends to keep the samples must use `aoe_channel_pinned` and release the pin when done. And a pin answers two questions for free, because both were computed at parse time: whether the samples are non-decreasing — the precondition of any binary search over x — and what their extent is, which is what autoscaling needs without rescanning a million samples per curve per replot.

11.4 Building against it

The core is built as a shared library with optimisation on and no dependency beyond the C++ standard library and the platform's mapping and threading calls. The export macro resolves to `__declspec(dllexport)` on Windows and to a default-visibility attribute elsewhere when building shared, and to nothing when building statically. The artefact is `aoecore.dll`, `libaoecore.dylib` or `libaoecore.so` according to platform. Because the header declares only C types, any language with a C foreign-function interface can drive it; the Python binding in this repository is one client rather than a privileged one.

CHAPTER 12

The Python scripting API

Everything the interface can read, a script can read, through the same core and with the same performance. Importing the package is deliberately cheap: the exports are lazy, so importing `aoeviewer` pulls in neither NumPy nor the core library, which is what lets `-version`, `-help` and `-selftest` work on a machine where the core has not been built.

```
from aoeviewer import Session, FileHandle, SimulationHandle, IsoGroup, CoreError
```

12.1 The one rule

Arrays returned by `channel()` are **read-only views of C++ memory**, not copies. They are fast because of that. They carry a pin object as their NumPy base, which keeps the underlying block alive for as long as any view exists — so the memory cap can never free a buffer a script is still reading from — but they stop being meaningful when the owning file is reloaded or removed. Copy explicitly if you need to outlive that.

12.2 Session

A `Session` is created with no arguments, raises `CoreError` if the core cannot be initialised, and is a context manager. Its `last_error` property exposes the core's message.

An absent channel yields an empty array rather than an error, and callers should treat that as *nothing to plot*: different files in one study legitimately hold different channels. In the `stats` dictionary, `ranged` is true only when the x window was actually applied — a run whose x channel is shorter than its y channel cannot be windowed, and the figures are then whole-channel.

12.3 Handles

A `FileHandle` carries its `index` and exposes `name`, `path`, `headers`, `units` — parallel to the headers, with empty strings where the dialect carries none, so the two can always be zipped — `simulation_count` and `simulations`. It can `rename(name)`, which changes the display name only, and `reload()`. The simulation list is cached on the session, because building one handle costs three ABI calls and a thousand-run file would otherwise cost three thousand on every access.

A `SimulationHandle` carries `file_index`, `index`, `sim_id` — what the file calls it — `name`, which is what the user sees, and `key`, the identity styles attach to, which never changes. It can `rename(name)` and `channel(name)`.

An `IsoGroup` is a named interval: `name`, `low`, `high`, `active`, and `contains(value)`.

12.4 A worked script

```
import numpy as np
from aoeviewer import Session, CoreError

with Session() as session:
    handle = session.add_file("study/sweep.aoe")

    # A derived channel, checked before it is relied on.
```

Method or property	Meaning
`add_file(path) -> FileHandle`	opens a result file; raises `CoreError` on failure
`remove_file(index)`, `clear()`	unload one file, or all of them
`file_count`, `files`, `file(index)`	the open files
`stale_files() -> List[int]`	indices of files changed on disk since they were read
`parameters -> List[str]`	every name: headers, computed channels and aliases
`set_alias(alias, target)`, `remove_alias(alias)`, `resolve(name)`	aliases
`unit(name)`, `label(name)`	the unit, and `"ALTITUDE [ft]"` — one place decides how a channel is labelled
`set_computed(name, expr)`	defines a computed channel; raises `CoreError` carrying the parse message
`remove_computed(name) -> List[str]`	the name plus every dependant removed
`is_computed(name)`, `computed_source(name)`	interrogate a definition
`validate_formula(expr) -> (ok, message)`	parses and checks names, evaluates nothing
`channel(file, sim, name) -> ndarray`	read-only zero-copy view; an *empty* array when the channel is absent
`preload(file, sims, names)`	turns N lazy single-column loads into one batched, multi-threaded read; an empty `sims` means all
`decimate(x, y, max_points, x_range=None)`	min/max bucketing, at most 2×`max_points` points
`Session.bounds(values)`	static; extent ignoring NaN and infinity
`Session.nearest_index(x, target)`	static
`Session.stats(y, x=None, x_range=None)`	static; a dict of `min`, `max`, `mean`, `std`, `first`, `last`, `count`, `ranged`
`resident_bytes`, `trim_memory(keep_bytes)`	the cache, and LRU eviction that is always safe because columns are re-read
`cache_enabled`, `cache_active(file)`, `cache_path(file)`	sidecar state
`build_cache(file)`, `discard_cache(file)`	write or delete a sidecar; building costs a full parse
`iso_values(file, name)`	one value per run, NaN where absent
`iso_distinct(name)`	sorted distinct values across every open file
`iso_auto_group(values, n)`, `iso_group_index(value, groups)`	interval arithmetic
`format_value(value)`	the same formatting the lists show

Table 12.1: The `Session` surface.

```
    ok, message = session.validate_formula("deriv(THETA, TIME)")
    if not ok:
        raise SystemExit(message)
    session.set_computed("PITCH_RATE", "deriv(THETA, TIME)")

    # One batched, multi-threaded read for everything we are about to touch.
    session.preload(handle.index, [], ["TIME", "ALPHA", "PITCH_RATE"])

    # Bucket the runs by their Mach number.
    machs = session.iso_distinct("MACH")
    groups = session.iso_auto_group(machs, 3)

    for sim in handle.simulations:
        value = float(session.iso_values(handle.index, "MACH")[sim.index])
        bucket = session.iso_group_index(value, groups)

        t = sim.channel("TIME")            # read-only view, zero copy
        q = sim.channel("PITCH_RATE")
        stats = Session.stats(q, t, (5.0, 25.0))   # over 5 s .. 25 s only
        if stats is None:
            continue
        print("{0}  MACH={1}  group={2}  peak q={3:.4g}  ranged={4}".format(
            sim.sim_id, session.format_value(value),
            groups[bucket].name if bucket >= 0 else "-",
            stats["max"], stats["ranged"]))

    # What a tile would actually draw for a 900 px wide plot.
    t = handle.simulations[0].channel("TIME")
    a = handle.simulations[0].channel("ALPHA")
    dx, dy = session.decimate(t, a, max_points=1800, x_range=(0.0, 60.0))

    print(session.label("ALPHA"), dx.size, "points drawn of", t.size)
    print("cache", session.resident_bytes // (1024 * 1024), "MB")
    session.trim_memory(256 * 1024 * 1024)
```

Two habits are worth adopting. Call `preload` once for everything a loop will touch, rather than letting each `channel()` trigger its own load — the batched read uses the whole thread pool and one mapping. And for a file of any size, call `build_cache()` once; the graphical application does this automatically above 8 MB, but a script gets no sidecar unless it asks.

Beyond the façade, `aoeviewer._core` exposes `library_path()`, the `Pin` type with its `sorted` flag and `bounds()`, `pin_of(array)` to recover the pin behind a view, the array adapters, and the module-level `VERSION` and `WORKER_COUNT`.

CHAPTER 13

Performance, limits and troubleshooting

13.1 Where the time goes

Three costs dominate, and each has its own mitigation. Parsing text into floats is paid once per column per run, in parallel, and is avoided entirely on the second and subsequent opens by the sidecar. Deciding what to draw is paid on every pan and zoom, and is bounded by the pixel budget rather than by the size of the data. Creating graphics items is paid when a tile's shape changes, and is minimised by concatenating curves that share a pen into one polyline and by adopting existing items whenever the data behind them is unchanged.

Limit	Value and meaning
Tiles per page	25, a 5×5 grid. Every page always holds 25 tile objects; shrinking a grid hides them rather than destroying them
Points per pixel	2 — one for each bucket's high and low
Points per tile	1 500 000 across all its curves, as a second ceiling
Legend rows	60, after which the remainder is summarised as … *and N more*
ISO value rows	400 per parameter in the tree; the ticks for the rest still apply
ISO values named in a tooltip	8, then *+N more*
Sample cache	2048 MB by default, floored at 64 MB, trimmed to 80% of the cap on overflow
Sidecar threshold	8 MB; below it, parsing is quick enough that a sidecar would save nothing measurable
Formula dependency depth	32, beyond which the definition is reported as cyclic
Redraw coalescing	25 ms, so a drag produces one resample at its end rather than one per pixel
Caption commit delay	400 ms of idle typing
File-change poll	3000 ms, skipped while a background task is running
Shutdown wait	30 s for background work to finish

Table 13.1: Every numeric limit in the program, in one place.

13.2 Getting the best out of it

Open each large file once and let the sidecar be written; the cost is paid on the first open and never again, and the status message says when it is happening. Prefer fewer, larger pages over many sparsely filled ones, since a fresh 5×5 page costs most of a second to build its widgets while switching between pages of the same shape merely redraws. Where a tile shows many runs of the same channel, turn the legend's **File** and **Simulation** parts off so identical rows merge — sixty rows of legend cost more to lay out than the curves do. And if the cache figure in the status bar is climbing towards the cap on a machine with plenty of memory, raise the cap with `-memory-limit`: eviction is safe, but a column that is evicted and re-read is a column parsed twice.

13.3 Troubleshooting

13.3.1 The program will not start

Run `python -m aoeviewer -selftest`. It checks each dependency separately and then the core, so the first `FAIL` names the problem. If the core is the failure, the error lists every directory

that was searched; build it, or point `AOEVIEWER_CORE` at the library. On Windows, a load failure that looks like a missing dependency suggests running `objdump -p` against the DLL to see what it wants. Whatever happens, the traceback is in `aoeviewer.log`, whose location is reported by `--version`.

13.3.2 A file opens but shows nothing

Opening succeeds as long as the file has a usable `*PARAMETRES` block, so a file with headers and no `*TABLEAU` opens with zero runs — the **DATA** tab will show the file with nothing beneath it. If runs are present but no curve appears, check in order: that runs are ticked in **DATA**; that no **ISO** filter is holding them back, which the status bar states explicitly; and that the tile has both an x channel and at least one y channel, which the tile's own title says in words.

13.3.3 The curves look wrong on a phase plane

A plot whose x channel is not monotonic — Mach against altitude over a climb and descent, for instance — needs the unsorted path through the decimator. The application always requests detection, so this is handled; but a script calling `decimate()` on such data should be aware that the sorted fast path would bracket only the first crossing. There is a related caveat in `nearest_index`: its fast path is chosen by comparing the first and last samples, which is a heuristic rather than a sortedness test, so a phase-plane channel that happens to end above where it started can return a wrong index.

13.3.4 A value I expected is missing or reads as *nan*

An unreadable token becomes not-a-number rather than zero, by design, so a gap in the file is a gap in the curve. Division by zero and the out-of-domain cases of `sqrt`, `asin`, `acos` and the logarithms also yield NaN rather than an infinity, because an infinity would destroy the plot's y range. In **INSPECT**, NaN prints as a dash.

13.3.5 A channel disappeared after I removed a computed parameter

Removing a computed parameter removes everything defined in terms of it, and the status message names what went. Re-create the dependency first, then the dependants.

13.3.6 Colours changed when I opened another file

They should not: a colour is allocated on first use and kept for the session, keyed on an identity that survives renaming. If they did change, something called **Re-assign palette**, which deliberately forgets the assignments so that the next redraw allocates afresh.

13.3.7 I reopened a workspace and the plot is empty

Both halves of what gets drawn are restored, so start by reading the status bar. If it says runs are held back by the ISO filters, open **ISO**: a filter whose node reads *no run satisfies it* is doing exactly what it was asked to and the data no longer contains anything in that band, so widen or untick it. A filter reading *not in the loaded files* is filtering nothing and is not the cause.

If the status bar says no runs are plotted at all, the **DATA** tab has nothing ticked — the workspace records which runs were chosen by index, so opening it against a file with fewer runs leaves the missing ones simply absent.

The dialog shown at load time, **Workspace loaded with differences**, lists everything the workspace asked for that this data could not give: unopenable files, formulas that could not be defined and why, aliases whose channel is gone, and both ISO cases. If it did not appear, everything in the workspace resolved.

APPENDIX A

Keyboard shortcuts and gestures

Keys or gesture	Action
Ctrl+L	Load workspace
Ctrl+S	Save workspace (overwrites the current file)
Ctrl+Shift+S	Save workspace as — asks, then continues in the new file
Ctrl+0	Auto-fit every tile
Ctrl+1 … Ctrl+4	Mouse mode: Select, Inspect, Pan, Zoom
Click a tile	make it the active tile
Double-click a tile	auto-fit that tile alone
Double-click a row in **DATA**	rename that file or run
Double-click a row in **COMPUTE**	load that definition for editing
Right-click in **DATA**	plot all or no runs of a file; rename; copy the full path
Right-click in **PARAMS**	create an alias; use as x axis; copy the name
Right-click in **ISO**	group, ungroup, select all or none, remove the filter
Drag in **Inspect**	measure a slope (shorter than 6 px counts as a click)
Drag a marker or band edge in **Select**	move it
Drag the legend	place it freely; the corner becomes *custom*
Drag a caption in **Select**	move it within its tile
Drag on a 3D tile in **Pan**	rotate
Middle-drag or Shift+drag on a 3D tile	translate
Right-drag or Ctrl+drag on a 3D tile	zoom
Wheel on a 3D tile in **Zoom**	zoom

Table A.1: Everything reachable from the keyboard or the mouse.

APPENDIX B

Environment variables

Variable	Effect
`AOEVIEWER_CORE`	Path to the core library, or to a directory containing it. Searched first, before every built-in location
`AOEVIEWER_LOG_DIR`	Directory for `aoeviewer.log`
`AOE_CACHE`	Set to `0`, `false`, `no` or `off` to disable sidecar caching entirely. Read once per process
`AOE_CACHE_DIR`	Overrides the per-user sidecar directory

Table B.1: Environment variables the program reads.

APPENDIX C

The workspace file

A workspace is UTF-8 JSON with the extension `.aoews`, written with two-space indentation.

```
{
  "version": 2,
  "files":     ["/abs/path/run_a.aoe", "..."],
  "computed": {"LOAD_FACTOR": "NZ * 9.80665"},
  "aliases":  {},
  "pages": [
    {"name": "Page 1", "rows": 2, "cols": 2, "tiles": [ ... ]}
  ],
  "active_page": 0,
  "selected_sims": [[0, 0], [0, 1]],
  "styles": {
    "colors":  {"sim::flight_a.aoe::SIM_001::0": "#3fa9f5"},
    "dashes":  {"param::ALPHA": "dash"},
    "symbols": {"param::ALPHA": "o"},
    "widths":  {"param::ALPHA": 2.0},
    "line_width": 1.6,
    "symbol_size": 6.0
  },
  "markers": [
    {"x_param": "TIME", "position": 5.0, "visible": true,
     "color": "#ff3333", "dot_size": 9.0}
  ],
  "areas": [
    {"x_param": "TIME", "x_min": 10.0, "x_max": 20.0,
     "color": "#3380ff", "alpha": 0.2, "visible": true}
  ],
  "global_x_limits": {"TIME": [0.0, 60.0]},
  "global_y_limits": {"ALPHA": [-5.0, 20.0]},
  "dark_mode": true,
  "mouse_mode": 0
}
```

Listing C.1: The shape of a workspace file.

Each entry of `"tiles"` carries the complete tile state listed in the table in Chapter 5, including its captions. Only the *visible* tiles of a page are written; loading pads the list back up to twenty-five.

The loader sets only the attributes it recognises and ignores the rest, which is how workspaces written when markers and bands still carried a caption field continue to load, and how the withdrawn legend field is tolerated. The `"version"` key is written but never read.

`value_enabled` is a list of pairs rather than an object because JSON object keys are strings: round-tripping `0.75` through an object hands back `"0.75"`, which would silently stop matching the value it is about. Non-finite keys are dropped, so the file stays portable JSON.

Not saved: the active tile index, the tile clipboard, the memory limit, the window geometry, and the hidden tiles of a shrunken grid.

APPENDIX D

Defaults at a glance

Setting	Where	Default
Window size	startup	1500 × 920
Theme	**RAIL** foot	dark
Mouse mode	toolbar	Select
Grid	page bar	1 × 1
Legend, Grid	**STYLE**	on
Log Y, Points	**STYLE**	off
Colour by	**STYLE**	Simulation
First three curve colours	**STYLE**	red, blue, dark green
Line width	**STYLE**	1.6 (range 0.1–10.0)
Marker size	**STYLE**	6.0 (range 1.0–20.0)
Legend corner	**STYLE**	top-left
Legend parts	**STYLE**	Parameter only
Axis locks	**SCALE**	off
ISO groups	**ISO**	3
Azimuth, tilt	**3D**	45°, 25°
3D triad, 3D grid	**3D**	on
3D zoom half-width	**3D**	0.95
Band opacity	**AREA**	0.20 (range 0.02–1.0)
Band colour	**AREA**	`#3380ff`
Marker dot size	**MARKERS**	9.0 (range 2.0–30.0)
Marker colour	**MARKERS**	`#ff3333`
Caption size	**TEXT**	10.0 pt (range 4–72)
Caption position	**TEXT**	0.05, 0.05
Memory limit	command line	2048 MB (floor 64 MB)

Table D.1: Defaults and ranges.

APPENDIX E

Known limitations

The following are real shortcomings of this version, gathered here so they are easy to find. Four entries that appeared here previously — ISO filters and aliases missing from the workspace, the undocumented `ln`, and an ISO filter on an absent parameter hiding every run — have been fixed and are described in §4.5, Chapter 6 and Chapter 9.

The workspace version is not interpreted. A `"version"` of 2 is written and nothing reads it; there is no version-branching logic, so a future format change has no mechanism to fall back on.

Group boundaries differ by a hair between the two layers. The core treats every interval except the last as half-open; the interface helpers treat all of them as inclusive at both ends with the first claiming group winning. The two disagree only for a value sitting exactly on a shared edge.

`nearest_index` guesses at sortedness. Its fast path is chosen by comparing the first and last samples rather than by testing monotonicity, so a phase-plane channel that ends above where it started can return a wrong index.

`aoe_cache_path` writes shared state under a read lock. Two concurrent readers can invalidate each other's returned pointer. The Python layer is not exposed to it because `ctypes` copies the string immediately.

There is no cancellation. A running background task cannot be interrupted; the interface is disabled until it completes. What exists instead is coalescing — a newer queued request replaces an older one.

Markers and areas are not drawn on three-dimensional tiles. This is a deliberate restriction rather than an oversight, but it is a restriction.

Classic Mac line endings are not supported. Lines are split on the line feed, with carriage returns trimmed, so a file using lone carriage returns reads as a single line.